BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN TRÍ TUỆ NHÂN TẠO

Học phần AIP202 · 2 tín chỉ

XÂY DỰNG NỀN TẢNG AI EXAMHUB DAU HỖ TRỢ TẠO ĐỀ VÀ TỔ CHỨC THI TRỰC TUYẾN

Sinh viên thực hiện	:	**Họ và tên**	**Mã sinh viên**
		Sinh viên 1	**Đã ẩn danh**
		Sinh viên 2	**Đã ẩn danh**
Lớp	:	**23CT1**	
Ngành	:	**Công nghệ thông tin**	
Giảng viên hướng dẫn	:	**Giảng viên hướng dẫn**	
Năm học	:	**2026–2027**	

Đà Nẵng · 2026–2027

Lời cam đoan

Tôi xin cam đoan báo cáo này được thực hiện trong quá trình xây dựng và khảo sát hệ thống AI ExamHub DAU. Nội dung mô tả thiết kế, chức năng và kết quả kiểm thử được đối chiếu với mã nguồn tại thời điểm hoàn thành báo cáo. Các số liệu được nêu có căn cứ từ kết quả chạy thử; báo cáo không xem các giá trị cấu hình hoặc điểm tin cậy do chương trình trả về là kết quả đánh giá độ chính xác của mô hình.

Mọi số liệu, hình ảnh, mã nguồn và kết quả của tác giả khác được sử dụng trong đồ án đều được trích dẫn nguồn gốc rõ ràng trong phần tài liệu tham khảo. Toàn bộ phần mềm và thư viện được dùng đều có giấy phép cho phép sử dụng. Những phần có sử dụng công cụ trí tuệ nhân tạo hỗ trợ đã được nêu rõ trong báo cáo.

Nếu phát hiện có bất kỳ sự gian lận nào, tôi xin chịu hoàn toàn trách nhiệm trước Khoa và Nhà trường.

Đà Nẵng, ngày … tháng … năm …
Sinh viên thực hiện

Sinh viên 1 · Sinh viên 2

Lời cảm ơn

Em xin trân trọng cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Kiến trúc Đà Nẵng đã trang bị kiến thức nền tảng và tạo điều kiện để em thực hiện đồ án. Em cũng xin cảm ơn giảng viên hướng dẫn đã dành thời gian góp ý, giúp em định hướng việc phân tích yêu cầu, triển khai hệ thống và hoàn thiện báo cáo. Cuối cùng, em xin cảm ơn gia đình và bạn bè đã động viên em trong quá trình học tập và thực hiện đồ án.

Đà Nẵng, ngày … tháng … năm …
Sinh viên thực hiện

Sinh viên 1 · Sinh viên 2

Tóm tắt

AI ExamHub DAU là nền tảng web hỗ trợ giảng viên quản lý ngân hàng câu hỏi, tạo đề từ tài liệu học tập và tổ chức thi trực tuyến cho sinh viên. Hệ thống được xây dựng theo kiến trúc web gồm giao diện React và TypeScript, API Node.js/Express và cơ sở dữ liệu PostgreSQL. Tài liệu PDF, DOC/DOCX và TXT được tiếp nhận để trích xuất văn bản; khi cấu hình khóa dịch vụ, hệ thống có thể yêu cầu mô hình ngôn ngữ tạo câu hỏi, còn khi không có dịch vụ thì sử dụng bộ sinh dự phòng dựa trên nội dung nguồn. Giảng viên có thể chọn loại câu hỏi, độ khó, mức độ tư duy, rà soát căn cứ trích dẫn, lập quy trình duyệt đề và tạo phòng thi. Sinh viên tham gia bằng mã phòng, lưu và tiếp tục bài làm; hệ thống tự chấm câu trắc nghiệm và cho phép giảng viên chấm tự luận. Quá trình kiểm chứng gồm 35 bài kiểm thử API, đều đạt, cùng một lượt build TypeScript/Vite thành công. Đây là bằng chứng kiểm thử chức năng và build, không phải phép đo độ chính xác của AI: đồ án chưa có tập dữ liệu chuẩn hay benchmark độc lập để định lượng chất lượng câu hỏi, OCR hoặc hiệu năng. Kết quả là một nguyên mẫu tích hợp được quy trình tạo đề và thi, đồng thời làm rõ các phần cần tiếp tục đánh giá trước khi triển khai thực tế.

Từ khoá: tạo câu hỏi, xử lý tài liệu, thi trực tuyến, React, Express, PostgreSQL.

Mục lục

Lời cam đoan i

Lời cảm ơn ii

Tóm tắt iii

1 Mở đầu và cơ sở lý thuyết 1
- 1.1 Lý do chọn đề tài . . . 1
- 1.2 Mục tiêu của đồ án . . . 1
- 1.3 Câu hỏi nghiên cứu và tiêu chí thành công . . . 2
- 1.4 Phương pháp thực hiện . . . 2
- 1.5 Đóng góp và giới hạn đóng góp . . . 3
- 1.6 Phạm vi, đối tượng và giả định . . . 3
- 1.7 Bố cục báo cáo . . . 4
- 1.8 Cơ sở lý thuyết và công nghệ . . . 4
  - 1.8.1 Kiến thức nền tảng . . . 4
  - 1.8.2 Công cụ và thư viện . . . 8
  - 1.8.3 Tài liệu nền tảng và công nghệ liên quan . . . 8

2 Phân tích, thiết kế và triển khai 10
- 2.1 Phân tích bài toán và yêu cầu . . . 10
  - 2.1.1 Tác nhân và kịch bản sử dụng . . . 11
  - 2.1.2 Yêu cầu phi chức năng và tiêu chí nghiệm thu . . . 13
- 2.2 Thiết kế kiến trúc hệ thống . . . 13
  - 2.2.1 Phân rã thành phần . . . 14
  - 2.2.2 Luồng xử lý phân tích tài liệu . . . 14
  - 2.2.3 Luồng thi trực tuyến . . . 14
- 2.3 Thiết kế dữ liệu . . . 15
  - 2.3.1 Sơ đồ thực thể–liên kết . . . 15
- 2.4 Thiết kế mô hình hoặc cơ sở tri thức . . . 16
  - 2.4.1 Thiết kế phân quyền . . . 16
  - 2.4.2 Bảo vệ API và thành phần AI . . . 17
- 2.5 Cài đặt và thử nghiệm . . . 17
  - 2.5.1 Môi trường triển khai . . . 17

2.5.2 Các thành phần chính 20
2.5.3 Giao diện và các luồng chính 21
2.5.4 Kiểm thử 22

3 Kết quả, đánh giá và kết luận 24
3.1 Kết quả đạt được 24
3.2 Độ đo và bằng chứng kiểm thử 25
3.2.1 Ma trận kiểm thử chức năng 26
3.2.2 Diễn giải các loại kết quả 27
3.2.3 Thiết kế thực nghiệm đánh giá AI đề xuất 28
3.3 Bàn luận 29
3.3.1 Đánh giá theo các vai trò 30
3.3.2 Đánh đổi kiến trúc 30
3.4 Hạn chế 30
3.5 Kết luận và hướng phát triển 31
3.5.1 Kết luận 31
3.5.2 Bài học rút ra 31
3.5.3 Hướng phát triển 31

Tài liệu tham khảo 33

Phụ lục 35
3.6 Hướng dẫn cài đặt và chạy hệ thống 35
3.7 Tổng quan cơ sở dữ liệu 35
3.7.1 Quy ước đọc từ điển dữ liệu 36
3.8 Từ điển dữ liệu 36
3.8.1 Bảng users – tài khoản và vai trò 36
3.8.2 Bảng questions – ngân hàng câu hỏi 37
3.8.3 Bảng exams – đề thi 39
3.8.4 Bảng exam_questions – câu hỏi thuộc đề 40
3.8.5 Bảng exam_rooms – phòng thi 40
3.8.6 Bảng exam_room_exams – đề có thể phân cho phòng . . 41
3.8.7 Bảng exam_attempts – lượt dự thi 42
3.8.8 Bảng exam_attempt_events – sự kiện lượt thi 43
3.8.9 Bảng submissions – bài nộp 44
3.8.10 Bảng results – kết quả chấm 45
3.8.11 Bảng rubrics – tiêu chí chấm 45
3.8.12 Bảng classroom_rooms – lớp học trực tuyến 46
3.8.13 Bảng classroom_members – thành viên lớp 47

3.9 Danh mục API theo nhóm nghiệp vụ 48
3.9.1 Xác thực, tài khoản và ngân hàng câu hỏi 49
3.9.2 Tài liệu và quản lý đề thi 50
3.9.3 Phòng thi, bài nộp và lớp học 51
3.10 Danh mục migration dữ liệu 52
3.11 Ma trận kiểm thử bổ sung 53
3.12 Rủi ro, giới hạn và biện pháp giảm thiểu 55
3.13 Mục tự kiểm trước khi nộp 56
3.14 Nhận xét của giảng viên hướng dẫn 57

Danh sách hình vẽ

1.1 Quy trình khai thác tài liệu và rà soát câu hỏi trong hệ thống. . . 7

2.1 Quan hệ giữa các vai trò và nhóm chức năng ở mức use case. . . . 12
2.2 Kiến trúc logic và luồng xử lý chính của AI ExamHub DAU. . . . 14
2.3 Sơ đồ quan hệ giữa các nhóm bảng nghiệp vụ chính. 16
2.4 Sơ đồ triển khai logic và các kết nối chính. 20
2.5 Wireframe khái niệm cho các khu vực của quy trình tạo đề. . . . 22

Danh sách bảng

1.1 Phương pháp và bằng chứng tương ứng với câu hỏi của đồ án. . . 3
1.2 Công nghệ được khai báo trong các manifest và cấu hình triển khai. 8

2.1 Nhóm yêu cầu chức năng của hệ thống. 11
2.2 Đặc tả các kịch bản sử dụng chính. 12
2.3 Yêu cầu phi chức năng và mức xác minh hiện có. 13
2.4 Các thực thể chính trong cơ sở dữ liệu. 15
2.5 Phân chia trách nhiệm theo thư mục mã nguồn. 18
2.6 Nhóm cấu hình cần thiết khi chạy hệ thống. 19

3.1 Đối chiếu mục tiêu và kết quả triển khai. 25
3.2 Kết quả xác minh phần mềm trong lần kiểm tra báo cáo. 26
3.3 Ma trận kiểm thử chức năng theo rủi ro nghiệp vụ. 27
3.4 Đề cương đo lường dành cho một đánh giá AI độc lập trong tương lai. 29

3.5 Từ điển dữ liệu bảng users. 37
3.6 Từ điển dữ liệu bảng questions. 38
3.7 Từ điển dữ liệu bảng exams. 39
3.8 Từ điển dữ liệu bảng exam_questions. 40
3.9 Từ điển dữ liệu bảng exam_rooms. 41
3.10 Từ điển dữ liệu bảng exam_room_exams. 42
3.11 Từ điển dữ liệu bảng exam_attempts. 43
3.12 Từ điển dữ liệu bảng exam_attempt_events. 44
3.13 Từ điển dữ liệu bảng submissions. 44
3.14 Từ điển dữ liệu bảng results. 45
3.15 Từ điển dữ liệu bảng rubrics. 46
3.16 Từ điển dữ liệu bảng classroom_rooms. 47
3.17 Từ điển dữ liệu bảng classroom_members. 48
3.18 Endpoint xác thực, người dùng và câu hỏi. 49
3.19 Endpoint xử lý tài liệu và vòng đời đề thi. 50
3.20 Endpoint tổ chức thi, chấm bài và quản lý lớp học. 51
3.21 Danh mục migration hiện diện trong dự án. 52
3.22 Kịch bản kiểm thử thủ công trước khi bảo vệ. 54
3.23 Sổ đăng ký rủi ro của nguyên mẫu AI ExamHub DAU. 55

1

Mở đầu và cơ sở lý thuyết

MỤC TIÊU CHƯƠNG

Chương này nêu bối cảnh, mục tiêu và phạm vi đồ án, sau đó trình bày cơ sở lý thuyết cùng công nghệ của nền tảng AI ExamHub DAU.

1.1 Lý do chọn đề tài

Trong quy trình đào tạo, giảng viên cần biên soạn câu hỏi, tạo nhiều đề, quản lý lịch thi, theo dõi bài làm và tổng hợp kết quả. Khi các công việc này thực hiện rời rạc, việc tái sử dụng học liệu và theo dõi trạng thái kiểm tra trở nên khó khăn. AI ExamHub DAU được xây dựng như một nền tảng thống nhất cho ngân hàng câu hỏi, hỗ trợ sinh câu hỏi từ tài liệu, quy trình duyệt đề và tổ chức lớp học, phòng thi trực tuyến.

Đề tài phù hợp với học phần trí tuệ nhân tạo vì kết hợp một ứng dụng thực tiễn với thành phần xử lý ngôn ngữ: đọc nội dung học liệu, đề xuất câu hỏi và liên kết câu hỏi với bằng chứng trong nguồn. Hệ thống cũng có phương án dự phòng dựa trên quy tắc khi không cấu hình dịch vụ mô hình ngôn ngữ. Việc khảo sát đồng thời phần AI và quy trình web giúp làm rõ giới hạn của một nguyên mẫu tích hợp, thay vì đồng nhất việc kết nối API với một mô hình đã được huấn luyện và đánh giá độc lập.

1.2 Mục tiêu của đồ án

Đồ án hướng tới các mục tiêu sau:

- Xây dựng ứng dụng web gồm giao diện người dùng, API backend và cơ sở dữ liệu quan hệ.
- Hỗ trợ tải tài liệu học tập, trích xuất văn bản và đề xuất câu hỏi trắc nghiệm/tự

luận có thông tin căn cứ.

- Hỗ trợ quản lý câu hỏi, tạo và duyệt đề thi, tổ chức phòng thi trực tuyến và lớp học theo vai trò.
- Kiểm tra các luồng chính bằng kiểm thử API và xác nhận giao diện có thể build/type-check.
- Nêu rõ các giới hạn của phần AI, dữ liệu và kiểm thử, không suy diễn kết quả chức năng thành độ chính xác mô hình.

1.3 Câu hỏi nghiên cứu và tiêu chí thành công

Phạm vi đồ án được định hướng bởi ba câu hỏi. Thứ nhất, làm sao kết nối việc tạo đề, duyệt đề và tổ chức kiểm tra vào cùng một quy trình phần mềm có phân quyền? Thứ hai, ở mức một nguyên mẫu, làm thế nào hỗ trợ khai thác tài liệu học tập mà vẫn giữ được khả năng giảng viên rà soát nguồn? Thứ ba, những bằng chứng nào là đủ để kết luận về độ hoàn thiện phần mềm, và những câu hỏi nào cần một nghiên cứu đánh giá AI riêng?

Tiêu chí thành công của phần mềm là các chức năng trọng tâm có API/UI tương ứng, quyền truy cập được phân theo vai trò, dữ liệu chính được lưu trong PostgreSQL, các luồng nghiệp vụ có kiểm thử tự động và frontend build thành công. Với thành phần AI, tiêu chí của nguyên mẫu chỉ là tạo được đầu ra có cấu trúc hoặc nhánh dự phòng và cung cấp thông tin nguồn để kiểm tra. Độ đúng, tính sư phạm, độ khó và hiệu quả học tập không được xem là đã xác nhận cho đến khi có bộ chuẩn đánh giá độc lập.

1.4 Phương pháp thực hiện

Đồ án sử dụng phương pháp phát triển theo hướng tích hợp và kiểm chứng nghiệp vụ. Trước hết khảo sát cấu trúc mã nguồn và schema, xác định các vai trò cùng luồng sử dụng; tiếp đó mô tả kiến trúc client–server, mô hình dữ liệu và hợp đồng API. Sau khi hiện thực frontend, backend và các migration, các luồng được xác minh bằng kiểm thử API hiện có và lệnh build của TypeScript/Vite. Cuối cùng, kết quả được phân loại thành bằng chứng chức năng, bằng chứng build và các đánh giá chưa có dữ liệu hỗ trợ.

Phương pháp này phù hợp với mục tiêu tạo nguyên mẫu tích hợp trong phạm vi đồ án, nhưng không thể thay thế thực nghiệm có người dùng, đo tải, kiểm thử

bảo mật độc lập hoặc đánh giá chất lượng nội dung AI bởi chuyên gia. Phạm vi và loại bằng chứng được tóm tắt trong Bảng 1.1.

Bảng 1.1. Phương pháp và bằng chứng tương ứng với câu hỏi của đồ án.

Câu hỏi	Cách khảo sát	Bằng chứng có/không có
Tích hợp nghiệp vụ	Đọc frontend, route API, middleware và schema; chạy luồng kiểm thử.	Mã nguồn, 35 ca API đạt và build frontend thành công.
Hỗ trợ khai thác tài liệu	Đọc xử lý trích xuất, cấu hình mô hình, kiểm tra đầu ra và fallback.	Có kiểm thử chức năng; chưa có chấm điểm câu hỏi bởi chuyên gia.
Mức độ chất lượng AI	Xác định tập chuẩn và độ đo cần thiết.	Chưa có tập gán nhãn; không báo cáo accuracy/F1/độ đúng nội dung.

1.5 Đóng góp và giới hạn đóng góp

Đóng góp chính là tích hợp được quy trình tạo câu hỏi có căn cứ, chọn ngân hàng để lập đề, duyệt và phát hành, tổ chức phòng thi, lưu tiến độ của sinh viên và ghi nhận kết quả trên cùng nền tảng. Về kỹ thuật, hệ thống minh hoạ cách phối hợp giao diện React/TypeScript, REST API Express và PostgreSQL, có Docker Compose cho môi trường phát triển. Thành phần AI được đóng góp ở mức tích hợp suy luận dịch vụ ngoài khi cấu hình cùng một nhánh sinh dự phòng; đồ án không tuyên bố đề xuất kiến trúc mô hình mới, huấn luyện mô hình riêng hay tạo ra một corpus học thuật mới.

1.6 Phạm vi, đối tượng và giả định

Đối tượng sử dụng gồm quản trị viên, giảng viên và sinh viên. Phạm vi triển khai khảo sát trong báo cáo tập trung vào ứng dụng web, API Node.js/Express và PostgreSQL. Luồng tạo câu hỏi nhận nội dung tài liệu PDF, DOC/DOCX hoặc TXT; khả năng trích xuất phụ thuộc vào định dạng và chất lượng văn bản/OCR. Mô hình ngôn ngữ bên ngoài chỉ được sử dụng khi nhà triển khai cung cấp cấu hình cần thiết; nếu không, hệ thống dùng bộ sinh dự phòng dựa trên văn bản.

Đồ án không xây dựng hay huấn luyện mô hình nền tảng riêng, không có bộ dữ liệu gán nhãn chuẩn để đo độ chính xác sinh câu hỏi, không triển khai các thư mục AI riêng như những dịch vụ độc lập, và chưa đánh giá tải hoặc bảo mật ở quy mô sản xuất. Chấm điểm tự động hiện tập trung vào câu trắc nghiệm; bài tự luận cần giảng viên chấm theo nội dung và tiêu chí.

Các giả định gồm: người tạo đề có quyền sử dụng tài liệu được tải lên; nhà triển khai tự cấu hình dịch vụ AI nếu muốn sử dụng nhánh mô hình ngôn ngữ; giảng viên xem xét câu hỏi/đáp án trước khi phát hành; cơ sở dữ liệu được khởi tạo theo schema/migration tương ứng. Những giả định này cần được rà soát khi đưa hệ thống vào sử dụng tại một cơ sở đào tạo thật.

1.7 Bố cục báo cáo

Phần còn lại của báo cáo gồm ba chương. Chương 1 trình bày cơ sở lý thuyết và công nghệ liên quan; Chương 2 phân tích yêu cầu, thiết kế dữ liệu và kiến trúc, đồng thời mô tả phần cài đặt; Chương 3 tổng hợp kết quả, bằng chứng kiểm thử, hạn chế và hướng phát triển. Sau phần tài liệu tham khảo là phụ lục hướng dẫn chạy hệ thống, từ điển dữ liệu và danh mục API.

1.8 Cơ sở lý thuyết và công nghệ

MỤC TIÊU CHƯƠNG

Phần này giới thiệu quy trình sinh câu hỏi từ tài liệu, các thành phần xử lý dữ liệu và công nghệ thực tế trong hệ thống.

1.8.1 Kiến thức nền tảng

Trong phạm vi đồ án, AI được dùng để hỗ trợ suy luận và sinh nội dung từ học liệu qua mô hình ngôn ngữ, không phải để huấn luyện một bộ phân lớp mới. Các khái niệm về tác tử và hệ thống trí tuệ nhân tạo được tham chiếu từ giáo trình nền tảng [18].

1.8.1.1 Sinh câu hỏi có điều kiện theo nguồn

Sinh câu hỏi từ tài liệu là quá trình chuyển nội dung văn bản thành câu hỏi và đáp án dựa trên các ý trong nguồn. Đầu vào của AI ExamHub DAU là nội dung đã trích xuất cùng môn học, số lượng, loại câu và phân bố độ khó/mức độ tư duy. Kết quả được chuẩn hoá thành các trường như nội dung câu hỏi, lựa chọn, đáp án,

giải thích và đoạn bằng chứng. Gắn bằng chứng nguồn giúp giảng viên đối chiếu đề xuất với học liệu trước khi lưu vào ngân hàng.

Khi có cấu hình khóa OpenAI, backend gửi yêu cầu đến dịch vụ mô hình ngôn ngữ. Đây là suy luận qua API, không phải mô hình do đồ án huấn luyện. Khi không có khóa hoặc phản hồi không dùng được, backend chuyển sang bộ sinh dự phòng dựa trên các câu/đoạn trong tài liệu. Bộ dự phòng giúp luồng có thể hoạt động mà không phụ thuộc dịch vụ ngoài, nhưng không đảm bảo chất lượng ngữ nghĩa tương đương người biên soạn.

1.8.1.2 Mô hình ngôn ngữ và sinh theo chỉ dẫn

Mô hình ngôn ngữ ước lượng chuỗi từ tiếp theo dựa trên ngữ cảnh đã nhận. Nhờ huấn luyện trên lượng văn bản lớn, một mô hình có thể làm theo chỉ dẫn và tạo đầu ra với nhiều dạng cấu trúc; nghiên cứu về mô hình ngôn ngữ few-shot mô tả khả năng điều kiện hoá đầu ra từ ví dụ trong ngữ cảnh [1]. Khả năng tạo câu tự nhiên không tự chứng minh nội dung là đúng. Mô hình có thể diễn đạt trôi chảy nhưng suy luận sai, bỏ qua điều kiện hoặc tự bổ sung chi tiết không có căn cứ.

Vì vậy, yêu cầu gửi đến mô hình cần làm rõ vai trò, nội dung nguồn, ma trận câu hỏi, định dạng đầu ra, quy tắc đáp án và yêu cầu trích nguyên văn đoạn bằng chứng. Ràng buộc cấu trúc giúp backend có thể phân tích JSON và từ chối đầu ra sai định dạng, nhưng một JSON hợp lệ vẫn có thể chứa kiến thức sai. Quy trình sản xuất cần có bước kiểm duyệt chuyên môn trước khi phát hành.

1.8.1.3 Sinh có truy xuất và khả năng truy nguyên

Một hướng nghiên cứu liên quan là kết hợp mô hình sinh với truy xuất tri thức bên ngoài. Retrieval-Augmented Generation (RAG) kết nối bộ sinh với kho tri thức để cung cấp thông tin và dấu vết nguồn cho câu trả lời [5]. Trong nguyên mẫu hiện tại, backend đưa nội dung tài liệu được người dùng gửi trực tiếp vào yêu cầu, rồi yêu cầu trả `sourceEvidence`; dự án chưa có kho embedding/vector, bộ truy hồi theo ngữ nghĩa hay pipeline RAG độc lập. Do đó, thuật ngữ "có căn cứ theo tài liệu" ở đây mô tả điều kiện prompt và trích dẫn nguồn, không khẳng định kiến trúc RAG đã được triển khai.

1.8.1.4 Độ khó và mức độ nhận thức

Độ khó của câu hỏi không chỉ phụ thuộc độ dài câu. Câu hỏi nhận biết/hiểu thường yêu cầu giải thích hoặc nhận diện khái niệm; câu vận dụng yêu cầu dùng quy tắc trong tình huống. Hệ thống lưu mức độ nhận thức `understanding` hoặc `application`

và các nhãn độ khó, đồng thời cho phép giảng viên đặt số lượng mong muốn. Các nhãn trong metadata là đầu vào của quá trình chọn câu hỏi, không phải kết quả đã được hiệu chuẩn theo một thang đo tâm lý lượng.

1.8.1.5 Chất lượng câu hỏi trắc nghiệm

Câu trắc nghiệm cần có một đáp án đúng duy nhất, phương án nhiễu cùng phạm trù và không vô lý, lời dẫn rõ ràng, tránh manh mối hình thức và tránh phụ thuộc vào kiến thức ngoài phạm vi môn học. Bộ sinh của dự án kiểm tra một số ràng buộc như số lượng/tính khác nhau của lựa chọn và khả năng đối chiếu bằng chứng. Những kiểm tra này không xác định đầy đủ tính đúng đắn chuyên môn hay chất lượng sư phạm, nên việc duyệt của giảng viên vẫn là một thành phần bắt buộc của quy trình sử dụng an toàn.

1.8.1.6 Trích xuất nội dung và OCR

PDF được đọc bằng `pdf-parse`; DOC/DOCX được chuyển qua Mammoth, vốn được thiết kế chủ yếu cho DOCX; TXT được đọc theo UTF-8. Backend cũng có nhánh OCR ảnh dùng Tesseract.js, khởi tạo với dữ liệu ngôn ngữ tiếng Anh. Giao diện hiện chỉ cho tải PDF, DOC/DOCX và TXT nên nhánh ảnh chưa phải luồng tải lên thông thường. Văn bản quét, DOC không tương thích, bố cục phức tạp hoặc tài liệu tiếng Việt đi qua bộ nhận dạng tiếng Anh có thể làm sai ký tự và ranh giới câu; nội dung trích xuất cần được rà soát trước khi dùng.

1.8.1.7 Mammoth và Tesseract.js

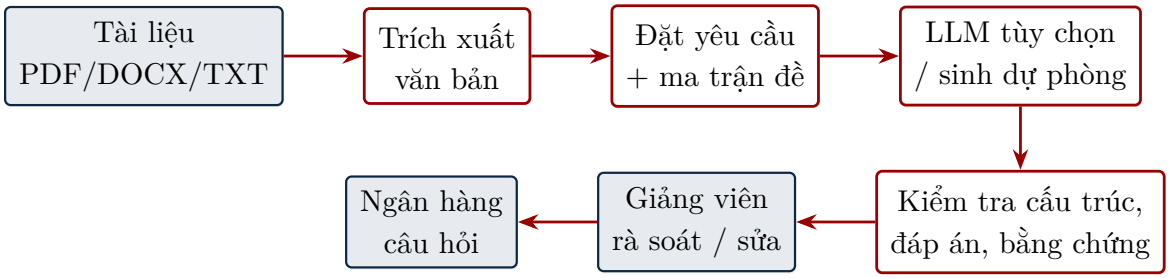
Mammoth tập trung vào việc chuyển DOCX sang HTML/văn bản, còn Tesseract.js cung cấp giao diện OCR JavaScript cho ảnh. Việc có thư viện trong backend không đồng nghĩa chất lượng trích xuất tiếng Việt đã được bảo đảm; chất lượng cần được đo riêng trên tập tài liệu phù hợp [22, 19].

1.8.1.8 Kiểm tra kết quả sinh

Kết quả mô hình không được đưa thẳng vào đề. Backend chuẩn hoá và kiểm tra các trường, loại câu, số lựa chọn, tính duy nhất của lựa chọn và đáp án, mức độ tư duy, độ khó, đồng thời đối chiếu bằng chứng với văn bản nguồn. Nếu kết quả không đạt điều kiện, hệ thống dùng kết quả dự phòng hoặc loại câu không hợp lệ. Đây là kiểm tra cấu trúc và một số ràng buộc nội dung; nó không thay thế việc giảng viên thẩm định kiến thức.

1.8.1.9 Mô hình rủi ro và trách nhiệm sử dụng

Khung quản trị rủi ro AI của NIST tổ chức hoạt động quanh các chức năng quản trị, lập bản đồ, đo lường và quản lý rủi ro [8]. Hướng dẫn UNESCO về AI tạo sinh trong giáo dục nhấn mạnh cách tiếp cận lấy con người làm trung tâm, độ tuổi phù hợp và bảo vệ dữ liệu cá nhân [20]. Từ góc nhìn hệ thống, rủi ro chính gồm câu hỏi sai đáp án, dữ liệu nguồn nhạy cảm bị gửi ra ngoài, OCR sai, lạm dụng tài khoản và phân quyền sai. Biện pháp đề xuất là giảm thiểu dữ liệu gửi đi, giới hạn quyền, không dùng đầu ra AI làm kết quả cuối cùng nếu chưa được rà soát, và ghi rõ dịch vụ bên ngoài có thể nhận nội dung tài liệu.



Hình 1.1. Quy trình khai thác tài liệu và rà soát câu hỏi trong hệ thống.

Hình 1.1 mô tả luồng logic; kết quả của bước kiểm tra tự động chỉ là cổng cấu trúc ban đầu. Phê duyệt nội dung vẫn thuộc trách nhiệm giảng viên.

1.8.2 Công cụ và thư viện

Bảng 1.2. Công nghệ được khai báo trong các manifest và cấu hình triển khai.

Thành phần	Phiên bản	Vai trò
React	18.3.1	Xây dựng giao diện web
TypeScript	5.6.3	Kiểm tra kiểu cho mã giao diện
Vite	5.4.11	Phát triển và đóng gói frontend
Node.js	20 (Docker)	Môi trường chạy backend
Express	5.1.0	Xây dựng REST API
PostgreSQL	16 (Docker)	Lưu trữ dữ liệu quan hệ
Tesseract.js	7.x	Thành phần OCR được khai báo trong backend
pdf-parse, Mammoth	Theo package.json	Đọc PDF và DOCX
Docker Compose	Theo môi trường cài đặt	Khởi chạy các container phát triển

1.8.3 Tài liệu nền tảng và công nghệ liên quan

Các mô hình ngôn ngữ hiện đại có thể sinh văn bản theo chỉ dẫn, nhưng đầu ra có thể chứa thông tin sai hoặc không bám sát nguồn. Vì vậy, hệ thống ứng dụng nguyên tắc có điều kiện theo tài liệu, yêu cầu bằng chứng và kiểm tra đầu ra trước khi giảng viên duyệt; tài liệu API mô hình là nguồn tham khảo cho cách tích hợp dịch vụ suy luận [9]. Nền tảng còn sử dụng các thành phần ứng dụng web và cơ sở dữ liệu đã được chuẩn hoá trong tài liệu chính thức của React, Express và PostgreSQL [6, 7, 21, 11, 17]. Khác với một nghiên cứu mô hình học máy, đồ án tập trung vào tích hợp các luồng nghiệp vụ và không đưa ra so sánh benchmark

giữa các mô hình.

1.8.3.1 Cơ sở dữ liệu quan hệ và tính toàn vẹn

PostgreSQL hỗ trợ khoá chính, khoá ngoại, ràng buộc và giao dịch, phù hợp với các quan hệ giữa tài khoản, đề, câu hỏi, lượt thi và bài nộp [17]. Câu hỏi và câu trả lời được biểu diễn dưới dạng JSONB ở một số cột để giữ lựa chọn và đáp án theo cấu trúc linh hoạt; những liên kết cần truy vấn và toàn vẹn tham chiếu vẫn có bảng liên kết riêng. Cách phân chia này tạo thuận lợi cho metadata, nhưng cần tránh để hai biểu diễn `question_ids` và `exam_questions` sai khác nhau.

1.8.3.2 Giao diện thành phần và hợp đồng HTTP

React tổ chức giao diện thành các component có trạng thái cục bộ và props [6]; TypeScript bổ sung kiểu dữ liệu tĩnh cho mô hình người dùng, câu hỏi, đề và lượt thi [7]. Vite cung cấp máy chủ phát triển và quy trình build tài nguyên frontend [21]. Backend dùng Node.js và Express để khai báo middleware, xác thực và tuyến HTTP [12, 11]. Những công nghệ này giảm công sức tích hợp, nhưng vẫn cần hợp đồng API nhất quán, xử lý lỗi và quyền truy cập ở backend.

1.8.3.3 Kiểm thử phần mềm và khả năng tái lập

Bộ kiểm thử backend sử dụng module `node:test` có sẵn trong Node.js; test runner cung cấp cách tổ chức và thực thi các test đồng bộ hoặc bất đồng bộ [13]. Compose mô tả dịch vụ backend, frontend và cơ sở dữ liệu trong một cấu hình để có thể dựng lại stack [2]. Kết quả kiểm thử chỉ có ý nghĩa đối với phiên bản mã, cấu hình và môi trường đã chạy; vì vậy khi tái lập cần ghi lại commit, phiên bản Node.js, biến môi trường đã che bí mật và tình trạng cơ sở dữ liệu.

2

Phân tích, thiết kế và triển khai

MỤC TIÊU CHƯƠNG

Chương này phân tích vai trò và yêu cầu, mô tả kiến trúc, dữ liệu, quy trình sinh câu hỏi và tổ chức thi, rồi trình bày cài đặt và thử nghiệm.

2.1 Phân tích bài toán và yêu cầu

Hệ thống có ba vai trò chính: quản trị viên quản lý tài khoản và dữ liệu toàn hệ thống; giảng viên quản lý câu hỏi, tài liệu, đề thi, lớp học và chấm bài; sinh viên tham gia lớp/phòng thi và xem lịch sử dự thi. Các yêu cầu chức năng trọng tâm được tóm tắt trong Bảng 2.1.

Bảng 2.1. Nhóm yêu cầu chức năng của hệ thống.

Mã	Chức năng	Mô tả
FR1	Tài khoản và phân quyền	Đăng nhập, hồ sơ người dùng và giới hạn chức năng theo vai trò.
FR2	Ngân hàng câu hỏi	Tạo, sửa, xoá, lọc và phân loại câu hỏi theo môn, loại và độ khó.
FR3	Hỗ trợ tạo đề	Đọc tài liệu, đề xuất câu hỏi, chọn phân bố, lưu đề và tạo mã đề.
FR4	Duyệt đề	Chuyển đề qua các trạng thái nháp, chờ duyệt, đã duyệt và phát hành.
FR5	Thi trực tuyến	Tạo phòng/mã tham gia, lưu câu trả lời, tiếp tục lượt thi và tự chấm trắc nghiệm.
FR6	Lớp học	Tạo lớp, tham gia bằng mã, quản lý trạng thái buổi học và thành viên.
FR7	Chấm và thống kê	Giảng viên chấm bài, ghi nhận phản hồi và xem số liệu theo quyền.

Các yêu cầu phi chức năng gồm bảo vệ API bằng xác thực và phân quyền, lưu dữ liệu bền vững qua PostgreSQL, kiểm tra đầu vào ở các luồng quan trọng, và hỗ trợ chạy các thành phần chính bằng Docker Compose. Báo cáo chưa xác nhận ngưỡng thời gian phản hồi hay số người dùng đồng thời; các tiêu chí đó cần được đo riêng trước triển khai.

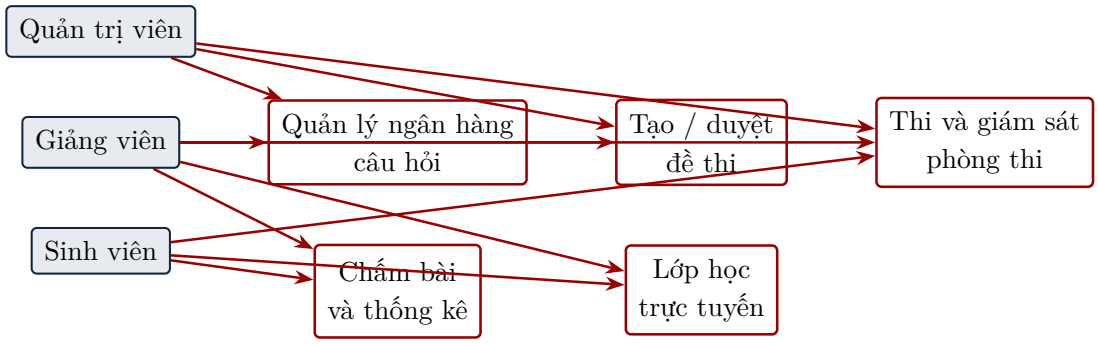
2.1.1 Tác nhân và kịch bản sử dụng

Các tác nhân tương tác với hệ thống qua giao diện web. Quản trị viên tạo và quản lý tài khoản, phân quyền và xem số liệu toàn hệ thống. Giảng viên nạp học liệu, xem các câu hỏi được đề xuất, hiệu chỉnh câu hỏi, lưu thành ngân hàng, tạo/duyệt đề, mở phòng thi, giám sát, chấm bài và xem phân tích. Sinh viên đăng nhập hoặc tham gia lớp/phòng thi theo quyền, trả lời câu hỏi, đánh dấu nội dung cần xem lại và nộp bài. Một lượt dự thi được gắn với phòng, đề và tài khoản sinh viên.

Bảng 2.2 mô tả một số kịch bản trọng yếu. Kịch bản thể hiện hành vi mong muốn của sản phẩm; các bước được triển khai đến đâu được đánh giá riêng trong Chương 3.

Bảng 2.2. Đặc tả các kịch bản sử dụng chính.

Kịch bản	Luồng chính	Kết quả / ngoại lệ cần xử lý
UC1 – Sinh câu hỏi	Giảng viên chọn tài liệu, môn học, số lượng, dạng câu và phân bố; gửi phân tích; xem ứng viên, bằng chứng và hiệu chỉnh.	Ứng viên có nguồn để đối chiếu; từ chối tài liệu rỗng, lỗi đọc hoặc đầu ra không hợp lệ.
UC2 – Tạo/phát hành đề	Chọn câu hỏi; khai báo thời lượng, thang điểm và metadata; lưu nháp; gửi duyệt; phê duyệt và phát hành.	Đề lưu kèm liên kết câu hỏi; lỗi ma trận không đủ câu phải được thông báo.
UC3 – Tổ chức thi	Giảng viên chọn đề, đặt thời gian, tạo mã; sinh viên nhập mã, làm bài, lưu và nộp.	Một sinh viên tiếp tục đúng lượt thi đã bắt đầu; bài nộp được tự chấm trắc nghiệm.
UC4 – Chấm tự luận	Giảng viên chọn bài đã nộp, nhập điểm trong giới hạn và phản hồi, lưu kết quả.	Kết quả gắn với bài nộp; không cho điểm vượt tổng điểm đề.
UC5 – Tham gia lớp	Giảng viên tạo lớp, chọn kiểu kết nối; sinh viên nhập mã; giảng viên mở/đóng buổi học.	Thành viên được lưu; lớp đã kết thúc hoặc đóng nhận thành viên thì từ chối lượt vào mới.



Hình 2.1. Quan hệ giữa các vai trò và nhóm chức năng ở mức use case.

Hình 2.1 chỉ thể hiện nhóm chức năng theo vai trò, không thay thế bảng endpoint hoặc chính sách phân quyền chi tiết.

2.1.2 Yêu cầu phi chức năng và tiêu chí nghiệm thu

Yêu cầu phi chức năng được phân thành an toàn, toàn vẹn dữ liệu, khả dụng, dễ vận hành và khả năng tiếp cận. Trạng thái đạt/đo được trình bày ở Bảng 2.3; các ngưỡng chưa đo không được xem là đã thoả mãn.

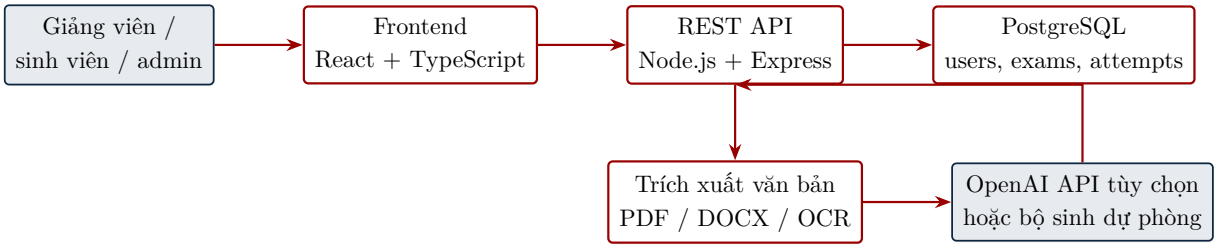
Bảng 2.3. Yêu cầu phi chức năng và mức xác minh hiện có.

Thuộc tính	Tiêu chí yêu cầu	Bằng chứng hiện có
Phân quyền	Mỗi thao tác ghi dữ liệu chỉ được phép với vai trò và chủ sở hữu thích hợp.	Có middleware và test phân quyền; chưa có kiểm thử độc lập đầy đủ.
Toàn vẹn	Khóa ngoại, ràng buộc, giao dịch tạo đề/phòng và tính duy nhất.	Có schema, migration và transaction ở các luồng chính.
Khả dụng	Lỗi dịch vụ ngoài không được biến thành kết quả thành công giả.	Có fallback cho sinh nội dung; cần kiểm tra log và UX ở triển khai thật.
Hiệu năng	Giao diện và API đáp ứng thời gian chấp nhận được ở tải mục tiêu.	Chưa có ngưỡng hoặc đo tải.
Khả năng tiếp cận	Điều khiển bằng bàn phím, nhãn biểu mẫu, trạng thái và tương phản phù hợp.	Chưa thực hiện đánh giá WCAG có hệ thống [23].
Quản lý dữ liệu	Hạn chế dữ liệu cá nhân/tài liệu gửi cho bên thứ ba; quy định lưu trữ và xoá.	Chưa có kiểm toán quyền riêng tư hoặc chính sách vận hành hoàn chỉnh.

2.2 Thiết kế kiến trúc hệ thống

Kiến trúc triển khai được trình bày ở Hình 2.2. Frontend React gửi yêu cầu HTTP đến REST API Express; backend áp dụng xác thực, nghiệp vụ và truy

vấn PostgreSQL. Luồng tài liệu đi qua bộ trích xuất văn bản, sau đó có thể gọi dịch vụ mô hình ngôn ngữ ngoài hoặc bộ sinh dự phòng. Các thư mục `ai-services/` và `ml-models/` hiện là khung tổ chức, không phải các dịch vụ độc lập đang được gọi trong luồng runtime.



Hình 2.2. Kiến trúc logic và luồng xử lý chính của AI ExamHub DAU.

2.2.1 Phân rã thành phần

Frontend đóng vai trò client hiển thị view theo vai trò và gọi các hàm trong lớp dịch vụ API. Backend tập trung xử lý các quy tắc miền, xác thực JWT, kiểm tra vai trò, thao tác cơ sở dữ liệu và tích hợp mô hình ngoài. PostgreSQL giữ dữ liệu nghiệp vụ; Docker Compose nối backend, frontend và cơ sở dữ liệu trong môi trường phát triển. Các module OCR/RAG/embedding trong cây thư mục dự án là vị trí quy hoạch cho tương lai, không đồng nghĩa các container AI này đã tồn tại trong cấu hình Compose hiện tại.

2.2.2 Luồng xử lý phân tích tài liệu

Luồng phân tích nhận `documentId`, môn học, số lượng, nội dung, ma trận loại câu, phân bố độ khó và mức độ nhận thức. Backend từ chối nội dung rỗng; nếu có nội dung, bộ phân tích tách các câu hỏi được định dạng sẵn, đồng thời gọi bộ sinh từ tài liệu. Các ứng viên được hợp nhất, chuẩn hoá và lọc theo ma trận. Những câu phù hợp được lưu vào ngân hàng câu hỏi trước khi trả về giao diện. Mô hình dữ liệu do đó phân biệt việc phân tích đề xuất với việc người dùng chọn câu để tạo đề hoàn chỉnh.

2.2.3 Luồng thi trực tuyến

Giảng viên tạo phòng từ một hoặc nhiều đề đã lưu. Mã phòng được tạo ngẫu nhiên; sinh viên dùng mã để bắt đầu lượt thi. Backend ghi một lượt cho cặp phòng–sinh viên, bộ câu hỏi được giao, đáp án, câu đánh dấu, vị trí hiện tại, thời hạn và thời điểm nộp. Khi nộp, bài thi trắc nghiệm được tự chấm; đề tự luận có thể yêu cầu

chấm riêng. Giảng viên có thể xem tiến độ và sự kiện chuyển tab được ghi lại. Sự kiện chuyển tab là tín hiệu hỗ trợ giám sát, không đủ để tự kết luận có gian lận.

2.3 Thiết kế dữ liệu

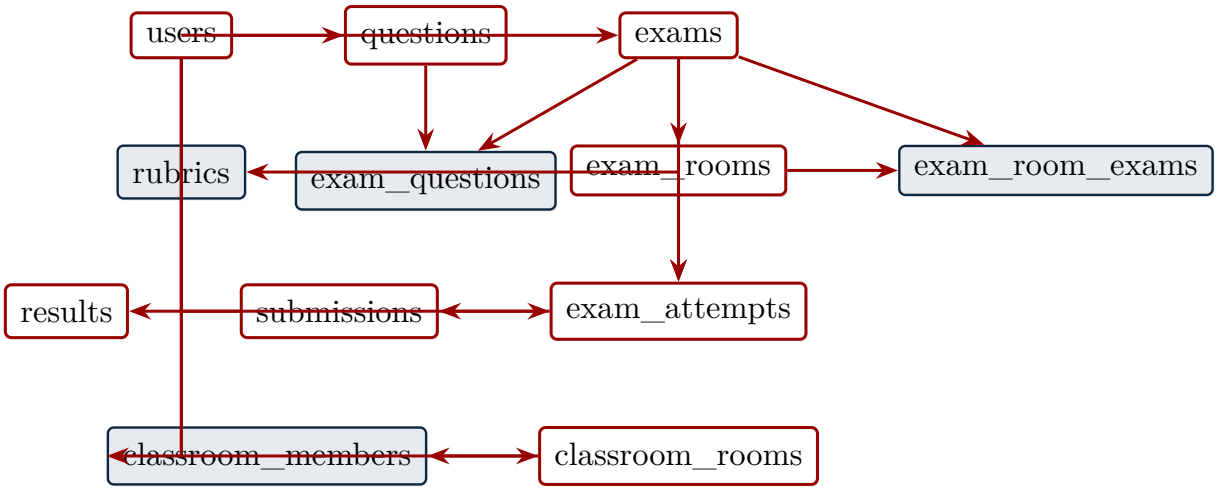
Hệ thống không sử dụng tập dữ liệu huấn luyện phân lớp. Dữ liệu nghiệp vụ gồm tài khoản, câu hỏi, đề thi, lượt dự thi, bài nộp, kết quả và lớp học. Các bảng lõi được mô tả khái quát ở Bảng 2.4; các trường chi tiết được khai báo trong lược đồ PostgreSQL của dự án.

Bảng 2.4. Các thực thể chính trong cơ sở dữ liệu.

Thực thể	Bảng	Vai trò
Tài khoản	users	Email, vai trò, hồ sơ và thông tin định danh.
Câu hỏi	questions	Nội dung, lựa chọn, đáp án, bằng chứng nguồn và rubric.
Đề thi	exams, exam_questions	Metadata đề và liên kết có thứ tự tới câu hỏi.
Thi trực tuyến	exam_rooms, exam_attempts	Phòng thi, câu hỏi được giao, đáp án và thời điểm.
Bài nộp/kết quả	submissions, results	Trạng thái nộp, điểm số và phản hồi.
Lớp học	classroom_rooms, classroom_members	Buổi học, mã tham gia và thành viên.

2.3.1 Sơ đồ thực thể–liên kết

Các quan hệ lõi trong schema được tóm tắt tại Hình 2.3. Người dùng tạo câu hỏi/đề/phòng/lớp; đề liên kết đến câu hỏi qua bảng nối; lượt thi thuộc về một sinh viên, một phòng và một đề; bài nộp liên kết với kết quả. Bảng lớp học nối người dùng sinh viên với phòng học bằng khoá ghép.



Hình 2.3. Sơ đồ quan hệ giữa các nhóm bảng nghiệp vụ chính.

2.4 Thiết kế mô hình hoặc cơ sở tri thức

Thành phần tạo câu hỏi có hai nhánh. Nhánh mô hình ngôn ngữ được gọi qua OpenAI API khi cấu hình `OPENAI_API_KEY`; yêu cầu đầu ra gồm loại câu, nội dung, lựa chọn, đáp án, giải thích và bằng chứng nguồn. Backend kiểm tra định dạng, ràng buộc lựa chọn và mức độ bám nguồn trước khi nhận câu. Nhánh dự phòng tách nội dung thành các đoạn/câu, nhận diện các mẫu kiến thức và tạo câu hỏi theo quy tắc; nhánh này giữ hệ thống hoạt động khi dịch vụ ngoài không sẵn sàng nhưng có phạm vi suy luận hạn chế.

Ở giai đoạn tạo đề, câu hỏi được lọc theo loại, độ khó và mức độ tư duy rồi lưu cùng metadata đề trong cơ sở dữ liệu. Khi tạo nhiều mã đề, backend xáo trộn thứ tự câu/lựa chọn và ánh xạ lại đáp án. Trong lúc thi, mỗi sinh viên được gắn với một bộ câu hỏi cho lượt thi, câu trả lời và vị trí hiện tại được lưu để hỗ trợ tiếp tục bài làm. Trắc nghiệm được tự chấm theo đáp án; tự luận được đưa qua quy trình giảng viên chấm và ghi phản hồi.

2.4.1 Thiết kế phân quyền

Chính sách quyền được áp dụng theo vai trò và một số trường hợp theo quyền sở hữu bản ghi. Admin quản lý dữ liệu toàn hệ thống; giảng viên thực hiện chức năng giáo viên và tài nguyên do mình quản lý; sinh viên chỉ thao tác với lượt thi/lớp mà mình được phép truy cập. Một token xác thực chỉ xác nhận danh tính; backend còn phải kiểm tra quyền của tài nguyên cụ thể. RFC 7519 định nghĩa cấu trúc JWT, nhưng việc sử dụng token an toàn phụ thuộc vào thuật toán, thời hạn, kiểm tra claim, bảo vệ khóa và thu hồi phiên [4]. Đăng nhập Google cần xác thực luồng

OAuth/OpenID Connect, kiểm tra trạng thái/redirect và liên kết tài khoản đúng theo danh tính đã xác minh [3].

2.4.2 Bảo vệ API và thành phần AI

OWASP Top 10 và OWASP API Security Top 10 nêu các nhóm rủi ro như kiểm soát truy cập sai, lỗi xác thực, cấu hình không an toàn và khai thác tài nguyên [14, 15]. Với mô hình ngôn ngữ, OWASP LLM Top 10 nhấn mạnh prompt injection, rò rỉ thông tin và đầu ra không được kiểm soát [16]. Ở hệ thống thi, chính sách thực tế cần kiểm tra quyền sở hữu từng đề/phòng, không trả đáp án đúng cho sinh viên trong khi thi, giới hạn kích thước/định dạng tải lên, lưu khóa API ở secret store, giảm thiểu nội dung nhạy cảm gửi ra mô hình và xem đầu ra mô hình như dữ liệu không tin cậy. Phân tích này là yêu cầu thiết kế và không phải kết quả kiểm thử xâm nhập.

2.5 Cài đặt và thử nghiệm

MỤC TIÊU CHƯƠNG

Phần này mô tả công nghệ, các thành phần đã triển khai và cách kiểm thử những luồng nghiệp vụ chính của hệ thống.

2.5.1 Môi trường triển khai

Dự án được tổ chức thành frontend web, backend API, cơ sở dữ liệu và cấu hình Docker Compose. Frontend dùng React 18 và TypeScript 5; backend dùng Node.js với Express 5; PostgreSQL 16 là dịch vụ cơ sở dữ liệu trong cấu hình container. Các phiên bản thư viện chính được liệt kê ở Bảng 1.2. Dockerfile của frontend dùng bước build Vite và Nginx để phục vụ tệp tĩnh; backend chạy trên ảnh Node.js Alpine.

Mã nguồn được kiểm tra tại thư mục cục bộ `E:\AI-EXAMHUB-DAU`. Cấu hình môi trường được cung cấp qua các tệp `.env.example`; khóa dịch vụ và mật khẩu dùng khi triển khai thực tế cần được thay bằng giá trị riêng, không đưa vào báo cáo hoặc kho mã nguồn. Hướng dẫn chạy từ mã nguồn được trình bày ở Phụ lục.

2.5.1.1 Cấu trúc mã nguồn

Các thành phần được tách theo trách nhiệm. `frontend/web/src/pages/` chứa các màn hình nghiệp vụ; `frontend/web/src/services/api.ts` là lớp gọi backend. `backend/api/src/app.js` khởi tạo ứng dụng Express và các

route; `backend/api/src/routes/` chứa các nhóm API. SQL nền tảng nằm trong `database/schema.sql`; các thay đổi theo phiên bản được đặt tại `database/migrations/`.

Bảng 2.5. Phân chia trách nhiệm theo thư mục mã nguồn.

Đường dẫn	Trách nhiệm trong hệ thống
`frontend/web/src/pages/`	Màn hình tạo đề, câu hỏi, thi, lớp học, người dùng, chấm và thống kê.
`frontend/web/src/services/api.ts`	Tạo yêu cầu HTTP, giữ kiểu dữ liệu API và quản lý thông tin xác thực ở phía client.
`backend/api/src/app.js`	Khởi tạo Express, tích hợp phân tích tài liệu và đăng ký route.
`backend/api/src/routes/`	Xác thực, câu hỏi, đề thi, phòng thi, lớp học, người dùng, bài nộp và phân tích.
`backend/api/src/middlewares/`	Xác thực JWT và kiểm tra vai trò.
`database/schema.sql`	Định nghĩa bảng, khoá, ràng buộc và chỉ mục chính.
`database/migrations/`	Chuyển đổi schema cho các phiên bản chức năng.
`docker-compose.yml`	Khai báo dịch vụ phát triển và mạng/kết nối giữa các container.

2.5.1.2 Biến cấu hình và phụ thuộc ngoài

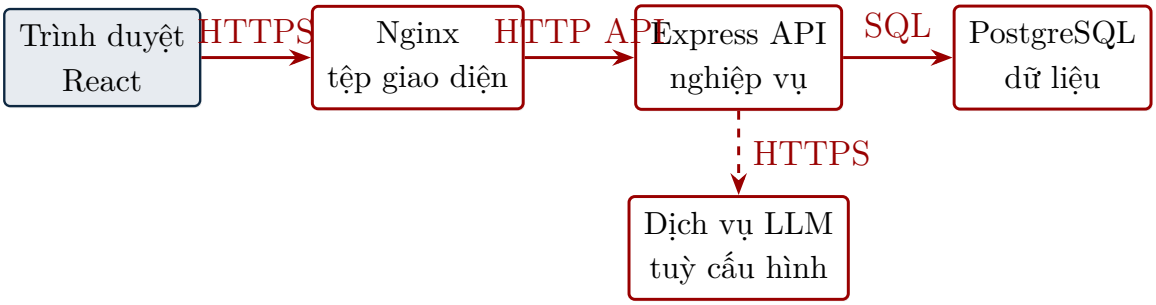
Bảng 2.6 mô tả nhóm cấu hình theo mục đích, không nêu giá trị bí mật. Ứng dụng có thể chạy phần sinh dự phòng mà không cấu hình API mô hình; chức năng Google OAuth cũng phụ thuộc thông tin client do nhà triển khai cấp. Hướng dẫn Node.js và Express được dùng để tham chiếu hành vi môi trường chạy và định tuyến [12, 11].

Bảng 2.6. Nhóm cấu hình cần thiết khi chạy hệ thống.

Nhóm biến	Ví dụ tên biến	Mục đích / lưu ý
API và xác thực	`PORT`, `JWT_SECRET`, `JWT_EXPIRES_IN`	Cổng API, ký và thời hạn token; secret phải duy nhất, không dùng giá trị dev khi triển khai.
Cơ sở dữ liệu	`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	Kết nối PostgreSQL; mật khẩu lấy từ môi trường an toàn.
Giao diện	`VITE_API_URL`	Địa chỉ gốc của API được đóng vào frontend.
LLM tùy chọn	`OPENAI_API_KEY`, `OPENAI_MODEL`	Gọi API sinh nội dung; khóa không được chuyển xuống trình duyệt.
Đăng nhập Google	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REDIRECT_URI`	Chỉ hoạt động khi được đăng ký/cấu hình đúng tại nhà cung cấp.
Nguồn gọi API	`CORS_ORIGIN`, `FRONTEND_URL`	Chính sách CORS và URL chuyển hướng OAuth.

2.5.1.3 Sơ đồ triển khai các thành phần

Hình 2.4 khái quát luồng kết nối của phiên bản hiện tại. Trình duyệt tải giao diện tĩnh do Nginx phục vụ, sau đó gửi yêu cầu đến API Express; backend truy cập PostgreSQL và chỉ gọi nhà cung cấp LLM bên ngoài khi đã cấu hình khóa. Các thành phần frontend, backend và cơ sở dữ liệu được khai báo trong Docker Compose [2]; mô hình không hàm ý dịch vụ AI riêng biệt chạy trong Compose.



Hình 2.4. Sơ đồ triển khai logic và các kết nối chính.

2.5.2 Các thành phần chính

2.5.2.1 Giao diện web

Ứng dụng React có màn hình đăng nhập, trang tổng quan, ngân hàng câu hỏi, trình tạo đề, duyệt đề, quản lý người dùng, chấm bài, phân tích, thi trực tuyến và lớp học. Giao diện gọi các hàm tập trung trong dịch vụ API ở `frontend/web/src/services/api.ts`. Một số trang điều phối luồng cụ thể gồm `ExamBuilderPage.tsx`, `OnlineExamPage.tsx`, `QuestionBankPage.tsx` và `GradingPage.tsx`.

2.5.2.2 API và nghiệp vụ

Backend Express tổ chức endpoint theo nhóm xác thực, người dùng, câu hỏi, đề thi, bài nộp, phân tích, phòng thi và lớp học. Middleware xác thực token và kiểm tra vai trò được dùng để giới hạn thao tác. Endpoint phân tích tài liệu tiếp nhận nội dung, gọi dịch vụ ngôn ngữ nếu cấu hình, kiểm tra đầu ra và trả các câu hỏi đề xuất kèm thông tin bằng chứng. Nhóm endpoint thi trực tuyến hỗ trợ tạo phòng, bắt đầu/tiếp tục lượt thi, lưu câu trả lời, nộp bài và giám sát tiến độ.

2.5.2.3 Lưu trữ và xử lý tài liệu

Lược đồ PostgreSQL lưu người dùng, câu hỏi, đề thi, phòng/lượt thi, bài nộp, kết quả và thành viên lớp. Các thay đổi lược đồ được quản lý bằng các tệp migration trong `database/migrations/`. Tệp tải lên được nhận trong bộ nhớ của backend để trích xuất nội dung; PDF dùng `pdf-parse`, DOC/DOCX được chuyển qua Mammoth và TXT được giải mã UTF-8. Backend có nhánh OCR ảnh dùng Tesseract.js với dữ liệu tiếng Anh, nhưng giao diện hiện không cung cấp đầu vào ảnh. Báo cáo không giả định tài liệu đã được lưu trữ lâu dài hay mọi tài liệu được trích xuất chính xác.

2.5.2.4 Quản lý vòng đời đề thi

Đề lưu trạng thái nghiệp vụ nháp, chờ duyệt, đã duyệt và phát hành cùng metadata như thời lượng, tổng điểm, năm học, học kỳ và ma trận câu hỏi. Khi lưu đề, backend kiểm tra số lượng, loại câu, điểm và thời lượng; các thao tác cần nhiều câu lệnh sử dụng transaction để tránh lưu dở dữ liệu giữa bảng đề và bảng liên kết. Câu hỏi thuộc đề được truy vấn qua `exam_questions` theo vị trí, thay vì dựa duy nhất vào một danh sách ID không có ràng buộc quan hệ.

2.5.2.5 Quản lý phiên làm bài

Khi sinh viên vào phòng, API tạo hoặc lấy lại lượt thi. API lưu đáp án theo ID câu hỏi, danh sách câu đánh dấu, chỉ số câu hiện tại, thời hạn và thời điểm nộp. Khi nộp, bài thi trắc nghiệm được tự chấm; đề tự luận có thể yêu cầu chấm riêng. Trước khi trả câu hỏi cho sinh viên, route loại các trường đáp án đúng, đáp án tham khảo và bằng chứng nội bộ. Cách tách thông tin đáp án là kiểm soát quan trọng vì ẩn đáp án chỉ ở giao diện không đủ để bảo vệ bài thi.

2.5.2.6 Thống kê và chấm điểm

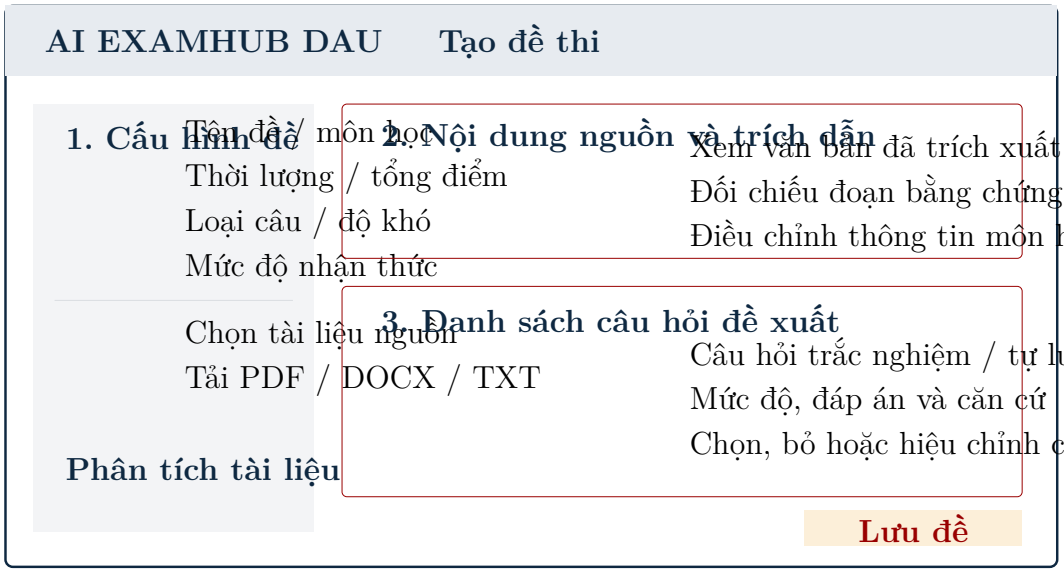
Trang thống kê hiển thị số liệu theo quyền người dùng; quản trị viên có phạm vi tổng thể, giảng viên tập trung dữ liệu liên quan đề/lớp của mình, còn sinh viên xem lượt dự thi. Chấm thủ công nhận điểm và phản hồi, sau đó lưu vào bảng `results`; điểm được kiểm tra không âm và không vượt tổng điểm của đề. Giao diện có nội dung mô tả "AI feedback", nhưng backend được kiểm tra trong đồ án không chứng minh đã có pipeline tự chấm tự luận bằng mô hình; báo cáo chỉ ghi nhận luồng chấm do giảng viên thực hiện.

2.5.3 Giao diện và các luồng chính

Các màn hình triển khai bao phủ hai nhóm luồng. Giảng viên có thể quản lý câu hỏi, phân tích tài liệu, chọn câu để tạo đề, gửi duyệt, phát hành và tạo phòng thi. Sinh viên đăng nhập, tham gia bằng mã, làm bài, lưu tiến độ, tiếp tục lượt thi và xem kết quả. Trang giám sát của giảng viên tải lại tiến độ theo chu kỳ; trang lớp học quản lý mã tham gia và danh sách thành viên. Báo cáo hiện không kèm ảnh chụp giao diện vì thư mục dự án không có ảnh chụp màn hình sản phẩm phù hợp để làm minh chứng.

2.5.3.1 Minh hoạ bố cục khu vực tạo đề

Hình 2.5 là sơ đồ bố cục khái niệm được dựng để giải thích các vùng chức năng, không phải ảnh chụp từ trình duyệt. Nó chỉ ra các bước người dùng cần quan sát: cấu hình đề, nạp nguồn, chọn câu hỏi và xác nhận trước khi lưu.



Hình 2.5. Wireframe khái niệm cho các khu vực của quy trình tạo đề.

2.5.3.2 Trạng thái thông báo và lỗi

Trong quy trình tạo tài liệu, trạng thái giao diện cần phân biệt đang tải, trích xuất xong, không có nội dung đọc được, không sinh được ứng viên, đã sinh được ứng viên và lỗi dịch vụ. Người dùng phải biết liệu câu hỏi thuộc nhánh LLM hay nhánh trích xuất dự phòng nếu hệ thống trả trường đó. Lỗi không nên hiển thị dưới dạng thông báo thành công, và thao tác thử lại không được tạo bản ghi trùng ngoài ý muốn. Những tiêu chí này được đưa vào ma trận kiểm thử đề xuất ở Phụ lục.

2.5.4 Kiểm thử

Backend có bộ kiểm thử tích hợp tại `backend/api/tests/api.test.js`. Trong lần chạy phục vụ báo cáo, toàn bộ 35 bài kiểm thử đạt, không có bài thất bại. Các trường hợp tập trung vào đăng nhập và phân quyền, CRUD tài khoản/câu hỏi/đề thi, phân tích tài liệu, sinh câu hỏi khi không cấu hình mô hình ngoài, phòng thi trực tuyến, tiếp tục bài làm, tự chấm trắc nghiệm và quản lý lớp học.

Frontend được kiểm tra bằng lệnh `npm run build`; TypeScript hoàn tất kiểm tra kiểu và Vite tạo gói production thành công. Kết quả này xác nhận build và các hành vi được kiểm thử, không phải đánh giá độ chính xác của AI, độ chính xác OCR, tải đồng thời hay thời gian phản hồi.

2.5.4.1 Tổ chức kiểm thử

Bộ kiểm thử API dùng Node.js test runner và chạy ứng dụng trên cổng động trong cùng tiến trình. Các test tạo tài khoản có vai trò, gửi HTTP request đến route và truy vấn PostgreSQL để xác minh dữ liệu. Phần lớn kiểm thử được tích hợp với database đã cấu hình trong môi trường này; do đó, người chạy lại cần database khả dụng và phải bảo đảm test không chạy nhầm vào dữ liệu production. Tài liệu của Node.js mô tả API test runner và cách khai báo test bất đồng bộ [13].

2.5.4.2 Kiểm soát an toàn dữ liệu

Hệ thống chứa thông tin định danh tài khoản, câu trả lời và lịch sử thi. Nguyên tắc cần áp dụng gồm chỉ truy xuất dữ liệu theo quyền sở hữu, tránh ghi token/khóa vào log, mã hoá đường truyền khi triển khai, sao lưu có kiểm soát và quy định thời hạn lưu dữ liệu. Khi gửi tài liệu cho nhà cung cấp LLM, cần thông báo rõ đối tượng dữ liệu, mục đích và cơ chế lưu/huỷ. Các nguyên tắc này phù hợp với khung quản trị AI và an ninh ứng dụng [8, 14, 15, 16]; chúng chưa được xem là đã xác minh nếu chưa có kiểm toán chuyên biệt.

3

Kết quả, đánh giá và kết luận

MỤC TIÊU CHƯƠNG

Chương này đối chiếu mục tiêu với sản phẩm đã triển khai, trình bày bằng chứng kiểm thử, thảo luận giới hạn và tổng kết hướng phát triển.

3.1 Kết quả đạt được

Các mục tiêu ở Chương 1 được đối chiếu tại Bảng 3.1. Sản phẩm hiện bao gồm ứng dụng web, API và lược đồ cơ sở dữ liệu; các luồng tạo câu hỏi, lập đề, duyệt đề, thi trực tuyến và lớp học có mã nguồn cùng kiểm thử. Thành phần tạo câu hỏi hỗ trợ gọi mô hình ngôn ngữ tùy cấu hình và có nhánh dự phòng. Các thành phần này chứng minh mức độ hoàn thiện chức năng của nguyên mẫu, không chứng minh chất lượng chuyên môn của câu hỏi do máy tạo.

Bảng 3.1. Đối chiếu mục tiêu và kết quả triển khai.

Mục tiêu	Bằng chứng trong dự án	Trạng thái
Ứng dụng web nhiều thành phần	React/TypeScript, Express API, PostgreSQL và Docker Compose.	Đạt
Hỗ trợ sinh câu hỏi từ học liệu	Trích xuất PDF/DOCX, tích hợp OpenAI tùy chọn, sinh dự phòng và kiểm tra đầu ra.	Đạt một phần
Quản lý ngân hàng và đề	CRUD câu hỏi/đề, chọn theo loại và độ khó, tạo mã đề, quy trình duyệt.	Đạt
Tổ chức thi và lớp học	Mã phòng/lớp, lưu lượt làm bài, giám sát, tiếp tục bài và tự chấm trắc nghiệm.	Đạt
Kiểm thử chức năng và build	35/35 bài kiểm thử API đạt; frontend build/type-check thành công.	Đạt
Đánh giá định lượng chất lượng AI	Chưa có tập chuẩn gán nhãn, phép đo chất lượng sinh hoặc benchmark OCR.	Chưa đạt

3.2 Độ đo và bằng chứng kiểm thử

Vì đồ án không huấn luyện mô hình và chưa có tập kiểm thử chuẩn cho câu hỏi hay OCR, không thể báo cáo *accuracy*, precision, recall, F1 hoặc độ chính xác OCR như kết quả thực nghiệm. Bằng chứng định lượng hiện có là số bài kiểm thử phần mềm và kết quả build được trình bày trong Bảng 3.2.

Bảng 3.2. Kết quả xác minh phần mềm trong lần kiểm tra báo cáo.

Hạng mục	Kết quả	Đánh giá
Kiểm thử API	35 bài; 35 đạt, 0 lỗi, 0 bỏ qua.	Đạt
Build frontend	TypeScript và Vite build thành công.	Đạt
Sinh câu hỏi không có LLM	Có bài kiểm thử xác nhận nhánh sinh dự phòng trả ứng viên.	Đạt theo kiểm thử chức năng
Chất lượng nội dung AI	Chưa đối chiếu với bộ câu hỏi chuẩn hoặc đánh giá chuyên gia có thang điểm.	Chưa đo
Hiệu năng và tải	Chưa có benchmark thời gian phản hồi hoặc tải đồng thời.	Chưa đo

Backend trả về trường `confidence` theo nhánh xử lý, nhưng giá trị này được gán trong mã nguồn và chưa được hiệu chuẩn bằng tập dữ liệu có nhãn. Do đó, báo cáo không diễn giải trường này thành xác suất đúng hay chỉ số đánh giá mô hình. Kiểm thử tự động tập trung vào cấu trúc đầu ra và nghiệp vụ API; giảng viên vẫn cần kiểm tra nội dung câu hỏi, đáp án và bằng chứng trước khi đưa vào đề thật.

3.2.1 Ma trận kiểm thử chức năng

Bảng 3.3 tóm tắt các nhóm hành vi được kiểm tra trong bộ test hiện có. Mục đích là chỉ ra loại lỗi có thể được phát hiện, không thay thế báo cáo từng lần chạy của CI. Tất cả 35 test hiện hữu đều được thực thi thành công trong lần xác minh. Các yêu cầu không xuất hiện trong test hiện hữu được giữ ở trạng thái chưa xác minh.

Bảng 3.3. Ma trận kiểm thử chức năng theo rủi ro nghiệp vụ.

Mã	Ca kiểm thử/điều kiện	Kết quả xác minh
T01	Health check và CORS preflight.	Đạt trong bộ test.
T02	Cập nhật vai trò người dùng; token hiện hành và quyền được làm mới.	Đạt trong bộ test.
T03	Sinh viên không truy cập phân tích quản trị; đăng ký công khai không tự cấp teacher/admin.	Đạt trong bộ test.
T04	CRUD tài khoản quản trị; phân trang và bulk import.	Đạt trong bộ test.
T05	Tạo, lưu và cập nhật đề; sinh mã đề với đáp án ánh xạ đúng.	Đạt trong bộ test.
T06	Phân tích tài liệu có câu hỏi/đáp án; dữ liệu rỗng bị từ chối.	Đạt trong bộ test.
T07	Không có LLM vẫn có ứng viên dự phòng; câu hỏi phải phù hợp văn bản nguồn.	Đạt theo ca hiện có.
T08	Câu hỏi trắc nghiệm có lựa chọn riêng biệt, đáp án và căn cứ nguồn.	Đạt theo ca hiện có.
T09	Phòng thi random đề, lưu bài, tự chấm trắc nghiệm và tiếp tục lượt.	Đạt trong bộ test.
T10	Lớp học: tạo, tham gia bằng mã, danh sách thành viên và kiểm soát quyền.	Đạt trong bộ test.
T11	OCR tiếng Việt, kiểm thử tải, tương phản/khả năng tiếp cận và trình duyệt thiết bị thật.	Chưa có bằng chứng.

3.2.2 Diễn giải các loại kết quả

Kiểm thử API kiểm chứng mã phản hồi, dữ liệu trả về và một phần trạng thái PostgreSQL. Kiểm thử tích hợp chạm vào nhiều thành phần cùng lúc, nên có thể phát hiện lỗi phối hợp mà test đơn vị của một hàm không phát hiện. Tuy vậy, các bài test được viết cho những kịch bản xác định, không bao phủ tất cả đầu vào, trình duyệt, nhà cung cấp OAuth hay cấu hình vận hành.

Lệnh build frontend xác nhận TypeScript không báo lỗi trong phạm vi project build và bundler đóng gói được các tài nguyên. Nó không đo khả năng sử dụng hoặc độ chính xác giao diện. Không có lượt test người dùng được tiến hành, do vậy báo cáo không tự đưa ra kết luận về mức độ hài lòng hoặc giảm thời gian biên soạn của giảng viên.

3.2.3 Thiết kế thực nghiệm đánh giá AI đề xuất

Để đánh giá chất lượng sinh câu hỏi trong tương lai, nên lập tập kiểm thử đại diện cho nhiều môn và định dạng tài liệu, sau đó giảng viên độc lập gán nhãn đúng/sai cho câu hỏi, đáp án, bằng chứng, độ rõ ràng, mức độ phù hợp và độ khó. Một phần mẫu nên được hai người chấm để tính mức nhất quán liên giám khảo; bất đồng cần có quy trình phân xử. Cần tách tài liệu đã dùng để chỉnh prompt khỏi tập đánh giá cuối cùng nhằm hạn chế rò rỉ thông tin vào phép đo.

Các câu hỏi trắc nghiệm có thể báo cáo tỷ lệ đáp án đúng được chuyên gia xác nhận, tỷ lệ bằng chứng hỗ trợ phát biểu, tỷ lệ lựa chọn hợp lệ và độ trùng lặp. Kết quả OCR có thể đo tỷ lệ lỗi ký tự và từ trên ảnh được gán nhãn; điểm số OCR không thay cho đánh giá hậu quả của một lỗi OCR lên câu hỏi sinh ra. Đánh giá phải phân tầng theo ngôn ngữ, loại tài liệu và chất lượng scan, đồng thời công bố số mẫu và khoảng tin cậy.

Bảng 3.4. Đề cương đo lường dành cho một đánh giá AI độc lập trong tương lai.

Khía cạnh	Cách đo đề xuất	Đối tượng đánh giá
Độ đúng nội dung	Tỷ lệ phát biểu/đáp án được chuyên gia xác nhận trên tập chưa dùng để phát triển.	Mỗi câu sinh và đáp án.
Bám nguồn	Tỷ lệ câu có bằng chứng nguyên văn, định vị đúng và đủ để hỗ trợ đáp án.	Cặp câu hỏi – nguồn.
Chất lượng sư phạm	Thang điểm rõ ràng, một đáp án đúng, distractor hợp lý, ngôn ngữ và mức độ nhận thức.	Giảng viên chấm độc lập.
Độ đa dạng	Tỷ lệ trùng lặp nội dung và phân bố ý/chủ đề trong tập câu hỏi.	Tập đề sinh từ nhiều tài liệu.
OCR	Character/Word Error Rate trên ảnh quét có transcript chuẩn.	Từng nhóm ngôn ngữ và chất lượng ảnh.
Độ trễ/chi phí	Thời gian p50/p95, lỗi/time-out và chi phí theo tài liệu/câu hỏi.	Pipeline và từng nhà cung cấp.

Mỗi độ đo cần được báo cáo theo phương pháp đã định trước; không gộp nhiều nhà cung cấp hoặc nhiều loại tài liệu thành một con số duy nhất nếu phân bố mẫu khác nhau. Trường `confidence` hiện được gán theo nhánh xử lý và chưa được hiệu chuẩn, nên không thể thay thế xác nhận chuyên gia hoặc khoảng tin cậy thống kê.

3.3 Bàn luận

Điểm mạnh của thiết kế là có thể chạy quy trình nghiệp vụ liên tục từ tài liệu và ngân hàng câu hỏi tới đề thi, lượt thi và kết quả; nhánh dự phòng cũng cho phép kiểm thử luồng sinh mà không phụ thuộc khóa dịch vụ ngoài. Việc lưu bằng chứng nguồn và hỗ trợ giảng viên rà soát là phù hợp với yêu cầu truy xuất nội dung, nhưng không loại bỏ hoàn toàn nguy cơ câu hỏi sai hoặc trích xuất sai.

Kết quả 35/35 bài đạt phản ánh phạm vi các ca kiểm thử đã viết. Nó không đồng nghĩa hệ thống không còn lỗi, cũng không đo độ chính xác nội dung do AI tạo. Build frontend thành công xác nhận khả năng biên dịch tại môi trường đã kiểm tra, nhưng chưa xác nhận triển khai lên máy chủ thực tế.

3.3.1 Đánh giá theo các vai trò

Đối với giảng viên, việc gộp ngân hàng, bộ lọc ma trận và lịch sử đề làm nền tảng cho khả năng tái sử dụng nội dung. Giá trị thực tiễn sẽ phụ thuộc vào độ chính xác của căn cứ và thời gian phải chỉnh sửa câu AI; đồ án chưa thực hiện đo trước/sau để kết luận tiết kiệm thời gian. Đối với sinh viên, lưu vị trí và cho tiếp tục bài làm hỗ trợ tính liên tục, còn mã truy cập tách việc dự thi khỏi việc gửi liên kết trực tiếp đến từng tài khoản. Đối với quản trị viên, quản lý vai trò tập trung thuận tiện nhưng làm tăng hậu quả nếu tài khoản có quyền cao bị chiếm dụng.

3.3.2 Đánh đổi kiến trúc

Tích hợp logic AI trong backend giúp sử dụng cơ sở dữ liệu và phân quyền đã có, giảm số dịch vụ phải vận hành. Mặt trái là tác vụ trích xuất/gọi LLM tốn thời gian có thể giữ request HTTP lâu và dùng bộ nhớ khi file được đọc trong memory. Việc tách AI thành worker hoặc hàng đợi có thể cải thiện độ bền và khả năng quan sát ở giai đoạn sau, nhưng đòi hỏi cơ chế trạng thái job, retry an toàn, idempotency và giới hạn chi phí; chưa nên xem là thành phần đã hiện thực.

3.4 Hạn chế

- Chưa có bộ dữ liệu chuẩn để đo độ đúng, độ bao phủ và độ đa dạng của câu hỏi sinh tự động.
- Chất lượng đầu ra phụ thuộc mô hình/API ngoài khi được cấu hình; bộ sinh dự phòng dựa trên quy tắc có thể tạo câu hỏi kém tự nhiên hoặc chưa đủ sâu.
- OCR chịu ảnh hưởng của chất lượng scan và bố cục; nhánh hiện tại dùng dữ liệu nhận dạng tiếng Anh, chưa có benchmark cho tài liệu tiếng Việt hoặc tập ảnh được gán nhãn.
- Câu tự luận cần được giảng viên chấm; giao diện có trường rubric và phản hồi nhưng báo cáo chưa xác nhận một bộ chấm tự động bằng AI.
- Chưa thực hiện kiểm thử tải, kiểm thử người dùng, đánh giá bảo mật triển khai và đo khả năng mở rộng.

- Các thư mục `ai-services/` và `ml-models/` mới là khung dự kiến; OCR và sinh câu hỏi hiện được gọi từ backend API.

3.5 Kết luận và hướng phát triển

MỤC TIÊU CHƯƠNG

Phần này tổng kết phạm vi hoàn thành, bài học rút ra và các hướng phát triển có thể kiểm chứng cho AI ExamHub DAU.

3.5.1 Kết luận

Đồ án đã khảo sát và xây dựng nguyên mẫu AI ExamHub DAU gồm frontend React, API Express và cơ sở dữ liệu PostgreSQL. Hệ thống hỗ trợ quản lý câu hỏi, tạo đề với trợ giúp từ nội dung tài liệu, duyệt đề, tổ chức phòng thi và lớp học trực tuyến, lưu tiến độ làm bài và chấm câu trắc nghiệm. Bộ kiểm thử API đạt 35/35 ca và frontend build thành công tại môi trường kiểm tra. Tuy vậy, chưa có benchmark độc lập để kết luận về độ chính xác của sinh câu hỏi, OCR hay hiệu năng; các đầu ra AI vẫn cần con người rà soát.

3.5.2 Bài học rút ra

Một thành phần AI hữu ích trong sản phẩm cần nhiều hơn một lời gọi mô hình: cần kiểm soát nguồn vào, chuẩn hoá cấu trúc, kiểm tra đầu ra và cung cấp cách để người dùng xác minh. Trong hệ thống tạo đề, việc lưu bằng chứng từ tài liệu và giữ quyền duyệt cuối cùng cho giảng viên là các bước quan trọng để hạn chế việc dùng nội dung không phù hợp.

Đồ án cũng cho thấy phải tách biệt kiểm thử chức năng với đánh giá mô hình. Một endpoint chạy đúng không chứng minh câu hỏi hay hoặc đáp án chính xác. Nếu tiếp tục thực hiện, cần lập bộ câu hỏi chuẩn, thống nhất tiêu chí chấm với giảng viên và công bố phương pháp đo trước khi cải thiện prompt hay bộ sinh dự phòng.

3.5.3 Hướng phát triển

- Xây dựng tập đánh giá có chuyên gia rà soát, đo mức bám nguồn, tính đúng của đáp án, độ rõ ràng, độ khó và mức trùng lặp.
- Hoàn thiện bộ xử lý OCR theo loại tài liệu, ngôn ngữ và chất lượng ảnh; lưu lại nguồn trích xuất để người dùng kiểm chứng.

- Thêm cơ chế duyệt/sửa câu hỏi AI, lịch sử phiên bản và chỉ đưa câu đã được phê duyệt vào ngân hàng chính thức.
- Đánh giá an toàn và quyền riêng tư khi gửi học liệu đến nhà cung cấp mô hình bên ngoài; cho phép cấu hình chính sách lưu trữ dữ liệu.
- Bổ sung kiểm thử tải, quan sát hệ thống và triển khai secrets an toàn; thay các giá trị phát triển bằng cấu hình vận hành phù hợp.
- Chỉ xem xét chấm tự luận có AI sau khi xác định rubric, bộ dữ liệu đối chiếu và quy trình giảng viên xác nhận kết quả.

Tài liệu tham khảo

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. https://arxiv.org/abs/2005.14165. Advances in Neural Information Processing Systems; 2020.

[2] Docker Inc. Docker compose documentation. https://docs.docker.com/compose/, 2026. Truy cập ngày 08/11/2026.

[3] Google. Openid connect. https://developers.google.com/identity/openid-connect/openid-connect, 2026. Tài liệu xác thực Google; truy cập ngày 08/11/2026.

[4] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, https://www.rfc-editor.org/rfc/rfc7519, 2015. IETF Standards Track; truy cập ngày 08/11/2026.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. https://arxiv.org/abs/2005.11401. Advances in Neural Information Processing Systems; 2020.

[6] Meta Platforms, Inc. React documentation: Learn react. https://react.dev/learn, 2026. Truy cập ngày 08/11/2026.

[7] Microsoft. The typescript handbook. https://www.typescriptlang.org/docs/, 2026. Truy cập ngày 08/11/2026.

[8] National Institute of Standards and Technology. Artificial intelligence risk management framework (ai rmf 1.0). https://doi.org/10.6028/NIST.AI.100-1. NIST AI 100-1, U.S. Department of Commerce; 2023.

[9] OpenAI. Chat completions api reference. https://platform.openai.com/docs/api-reference/chat, 2026. Tài liệu tham khảo cho tích hợp API; truy cập ngày 08/11/2026.

[10] OpenAPI Initiative. Openapi specification 3.1.1. https://spec.openapis.org/oas/v3.1.1.html, 2024. Truy cập ngày 08/11/2026.

[11] OpenJS Foundation. Express documentation: Routing. https://expressjs.com/en/guide/routing.html, 2026. Truy cập ngày 08/11/2026.

[12] OpenJS Foundation. Node.js api documentation. https://nodejs.org/docs/latest/api/, 2026. Truy cập ngày 08/11/2026.

[13] OpenJS Foundation. Node.js test runner. https://nodejs.org/api/test.html, 2026. Truy cập ngày 08/11/2026.

[14] OWASP Foundation. Owasp top 10:2021. https://owasp.org/Top10/2021/, 2021. Tài liệu nhận thức về rủi ro ứng dụng web; truy cập ngày 08/11/2026.

[15] OWASP Foundation. Owasp api security top 10 – 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/, 2023. Truy cập ngày 08/11/2026.

[16] OWASP Foundation. Owasp top 10 for large language model applications. https://genai.owasp.org/llm-top-10/, 2025. Truy cập ngày 08/11/2026.

[17] PostgreSQL Global Development Group. Postgresql 16 documentation. https://www.postgresql.org/docs/16/, 2026. Truy cập ngày 08/11/2026.

[18] Stuart Russell and Peter Norvig. Artificial intelligence: A modern approach. https://aima.cs.berkeley.edu/, 2021.

[19] Tesseract.js Contributors. Tesseract.js: Pure javascript ocr. https://github.com/naptha/tesseract.js, 2026. Mã nguồn và tài liệu thư viện OCR; truy cập ngày 08/11/2026.

[20] UNESCO. Guidance for generative ai in education and research. https://unesdoc.unesco.org/ark:/48223/pf0000386693, 2023. UNESCO; truy cập ngày 08/11/2026.

[21] Vite Contributors. Vite guide. https://vite.dev/guide/, 2026. Truy cập ngày 08/11/2026.

[22] Michael Williamson. Mammoth.js documentation. https://github.com/mwilliamson/mammoth.js, 2026. Thư viện trích xuất văn bản DOCX; truy cập ngày 08/11/2026.

[23] World Wide Web Consortium. Web content accessibility guidelines (wcag) 2.2. https://www.w3.org/TR/WCAG22/, 2023. W3C Recommendation; truy cập ngày 08/11/2026.

Phụ lục

3.6 Hướng dẫn cài đặt và chạy hệ thống

Mã nguồn của hệ thống nằm trong thư mục `E:\AI-EXAMHUB-DAU`. Cấu hình thật được đọc từ các tệp môi trường không nên đưa lên kho công khai. Để chạy stack phát triển bằng Docker Compose, tạo bản cấu hình cục bộ từ ví dụ và khởi động các dịch vụ:

Mã nguồn 3.1. Các lệnh khởi chạy stack phát triển.

```
cd E:\AI-EXAMHUB-DAU
docker compose up --build
```

Theo tệp Compose hiện có, frontend được công bố tại `http://localhost:8080`, backend tại `http://localhost:3000/api` và PostgreSQL tại cổng 5432. Docker Compose cho phép mô tả và khởi chạy nhiều container từ một cấu hình [2]. Các giá trị cổng và mật khẩu development trong dự án chỉ phục vụ cài đặt cục bộ; phải đổi trước khi triển khai có dữ liệu thật.

Để chạy từng phần, vào `backend/api`, cài phụ thuộc theo `package-lock.json`, chuẩn bị PostgreSQL và cấu hình các biến môi trường từ `.env.example`; sau đó chạy `npm run dev`. Frontend được chạy trong thư mục `frontend/web` bằng `npm install` và `npm run dev`; đặt `VITE_API_URL` trỏ tới backend đang chạy. Các lệnh kiểm tra là `npm test` tại backend và `npm run build` tại frontend. Nếu database schema chưa đầy đủ, cần áp dụng các migration theo đúng thứ tự trong thư mục dự án. Không chạy test bằng tài khoản hoặc database production.

3.7 Tổng quan cơ sở dữ liệu

Schema được khởi tạo từ `database/schema.sql` và bổ sung theo các migration. Định danh chính dùng UUID; các nội dung linh hoạt như lựa chọn, câu trả lời, danh sách ID hoặc metadata được lưu dạng JSONB; thời điểm nghiệp vụ dùng `TIMESTAMPTZ`. Các khoá ngoại quy định vòng đời phụ thuộc của bài nộp, kết quả, lượt thi và thành viên. Các quan hệ lõi được minh hoạ ở Hình 2.3; Bảng 2.4 trong Chương 2 là bản đồ thực thể ở mức khái quát.

3.7.1 Quy ước đọc từ điển dữ liệu

Trong các bảng dưới đây, PK là khoá chính; FK là khoá ngoại; UQ là giá trị duy nhất; NN là bắt buộc; CK là ràng buộc miền/điều kiện. Kiểu JSONB hỗ trợ dữ liệu JSON, nhưng không tự thay thế kiểm tra tính hợp lệ của cấu trúc payload trong API. Một số cột được bổ sung qua migration hoặc `ensureExamSchema()` lúc ứng dụng khởi động; vì vậy schema cuối cùng đang chạy phải được đối chiếu với toàn bộ migration, không chỉ một file SQL.

3.8 Từ điển dữ liệu

3.8.1 Bảng users – tài khoản và vai trò

Bảng `users` là gốc định danh của người dùng, giữ email đăng nhập, thông tin hồ sơ và phạm vi quyền. Vai trò được giới hạn vào `admin`, `teacher` hoặc `student`. Trường `institutional_id` hỗ trợ mã cán bộ/sinh viên và có chỉ mục duy nhất riêng khi trường khác NULL. Không lưu mật khẩu rõ; API lưu giá trị băm ở `password_hash`.

Bảng 3.5. Từ điển dữ liệu bảng users.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh tài khoản, mặc định sinh UUID.
email	VARCHAR(255)	NN, UQ	Địa chỉ đăng nhập, duy nhất.
password_hash	TEXT	NN	Mật khẩu đã băm; không phải mật khẩu gốc.
full_name	VARCHAR(255)	NN	Họ tên hiển thị.
role	VARCHAR(32)	NN, CK	Vai trò admin/teacher/student.
avatar_url	TEXT	NULL	Đường dẫn hoặc nội dung ảnh đại diện theo API.
phone	VARCHAR(30)	NULL	Số điện thoại liên hệ.
institutional_id	VARCHAR(64)	UQ, NULL	Mã định danh cơ sở đào tạo.
faculty	VARCHAR(255)	NULL	Khoa của người dùng.
department	VARCHAR(255)	NULL	Bộ môn hoặc đơn vị.
cohort	VARCHAR(64)	NULL	Khoá học.
class_name	VARCHAR(128)	NULL	Lớp hành chính.
created_at	TIMESTAMPTZ	NN	Thời điểm tạo tài khoản.

3.8.2 Bảng questions – ngân hàng câu hỏi

Bảng `questions` giữ nội dung và metadata cần cho việc chọn câu vào đề. Các trường liên quan loại câu, câu trả lời, mức nhận thức và nguồn có thể được mở rộng qua migration. Lựa chọn và rubric có cấu trúc linh hoạt; nội dung nguồn được lưu riêng

ở source_evidence để phục vụ rà soát và truy nguyên. Câu hỏi có thể tồn tại độc lập với người tạo nếu tài khoản gốc bị xoá.

Bảng 3.6. Từ điển dữ liệu bảng questions.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh câu hỏi.
subject	VARCHAR(255)	NN	Môn/chủ đề học thuật.
content	TEXT	NN	Lời dẫn hoặc nội dung câu hỏi.
choices	JSONB	NN	Danh sách lựa chọn; mặc định mảng rỗng.
answer_index	INTEGER	NULL	Chỉ số lựa chọn đúng cho câu trắc nghiệm.
answer_text	TEXT	NULL	Đáp án tham khảo dạng văn bản.
source_evidence	TEXT	NULL	Trích đoạn nguồn làm căn cứ cho câu hỏi.
cognitive_level	VARCHAR(32)	NULL	Mức nhận thức như understanding/application.
rubric	JSONB	NULL	Cấu trúc tiêu chí chấm tự luận nếu có.
difficulty	VARCHAR(16)	NN, CK	easy, medium hoặc hard; mặc định medium.
type	VARCHAR(32)	Schema mở rộng	Loại multipleChoice/essay; được bổ sung trong schema runtime/migration.
created_by	UUID	FK, NULL	Tài khoản tạo; xoá tài khoản đặt NULL.
created_at	TIMESTAMPTZ	NN	Thời điểm tạo câu hỏi.

3.8.3 Bảng exams – đề thi

Bảng `exams` lưu thông tin định danh và trạng thái vòng đời của đề. Các chi tiết cấu hình đề được lưu trong `metadata` dạng JSONB; các liên kết câu hỏi có thứ tự được chuẩn hoá ở bảng `exam_questions`. Schema cũng có `question_ids` nhằm tương thích với các luồng cũ. Khi bảo trì cần xác định một nguồn dữ liệu chuẩn để tránh hai biểu diễn bị lệch nhau.

Bảng 3.7. Từ điển dữ liệu bảng exams.

Trường	Kiểu	Ràng buộc	Ý nghĩa
`id`	UUID	PK	Định danh đề.
`title`	VARCHAR(255)	NN	Tên đề thi.
`subject`	VARCHAR(255)	NN	Môn học.
`status`	VARCHAR(16)	NN, CK	Trạng thái phát hành ở schema.
`workflow_status`	VARCHAR(16)	NN/default	Trạng thái quy trình: draft/review/approved/published trong ứng dụng.
`question_ids`	JSONB	NN	Danh sách ID tương thích với luồng legacy.
`metadata`	JSONB	NN	Thời lượng, tổng điểm, ma trận, năm học, rubric và thông tin đề.
`created_by`	UUID	FK, NULL	Tài khoản tạo; xoá người tạo đặt NULL.
`created_at`	TIMESTAMPTZ	NN	Thời điểm tạo.
`updated_at`	TIMESTAMPTZ	NN	Thời điểm cập nhật gần nhất.

3.8.4 Bảng exam_questions – câu hỏi thuộc đề

Bảng nối biểu diễn quan hệ nhiều–nhiều giữa đề và câu hỏi, đồng thời lưu thứ tự hiển thị. Một câu hỏi có thể tái sử dụng trong nhiều đề; xoá đề sẽ cascade xoá liên kết nhưng không xoá câu hỏi.

Bảng 3.8. Từ điển dữ liệu bảng exam_questions.

Trường	Kiểu	Ràng buộc	Ý nghĩa
exam_id	UUID	PK, FK	Đề chứa câu hỏi; xoá đề cascade liên kết.
question_id	UUID	PK, FK	Câu hỏi; không cho xoá nếu vẫn thuộc đề.
position	INTEGER	NN	Vị trí câu trong đề.

3.8.5 Bảng exam_rooms – phòng thi

Phòng thi gắn với một đề chính và có thể gắn nhiều đề bổ sung qua bảng exam_room_exams. Mã tham gia được đảm bảo duy nhất. Thời điểm mở, đóng và trạng thái điều khiển khả năng vào phòng.

Bảng 3.9. Từ điển dữ liệu bảng exam_rooms.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh phòng thi.
exam_id	UUID	FK, NN	Đề chính của phòng; xoá đề cascade phòng.
name	VARCHAR(255)	NN	Tên phòng.
starts_at	TIMESTAMPTZ	NULL	Thời điểm dự kiến bắt đầu.
ends_at	TIMESTAMPTZ	NULL	Thời điểm kết thúc/đóng nhận bài.
status	VARCHAR(16)	NN, CK	scheduled, active hoặc closed.
access_code	VARCHAR(8)	NN, UQ	Mã tham gia ngẫu nhiên duy nhất.
created_by	UUID	FK, NULL	Tài khoản tạo phòng.

3.8.6 Bảng exam_room_exams – đề có thể phân cho phòng

Bảng liên kết phòng với một hoặc nhiều đề được giảng viên chọn. Mỗi đề có vị trí trong danh sách; một lượt thi chọn đề được gán cho sinh viên và lưu ID đề ở exam_attempts.

Bảng 3.10. Từ điển dữ liệu bảng exam_room_exams.

Trường	Kiểu	Ràng buộc	Ý nghĩa
room_id	UUID	PK, FK	Phòng thi; xoá phòng cascade liên kết.
exam_id	UUID	PK, FK	Đề được đưa vào danh sách phân bổ.
position	INTEGER	NN, UQ theo phòng	Vị trí đề; duy nhất trong một phòng.

3.8.7 Bảng exam_attempts – lượt dự thi

Lượt thi lưu trạng thái làm bài độc lập với định nghĩa đề. Bộ câu hỏi được giao và đáp án lưu ở JSONB để tiếp tục bài làm; khoá duy nhất `(room_id, student_id)` bảo đảm một lượt chính cho sinh viên trong phòng theo schema hiện tại.

Bảng 3.11. Từ điển dữ liệu bảng exam_attempts.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh lượt thi.
room_id	UUID	FK, NN	Phòng thi chứa lượt.
exam_id	UUID	FK, NN	Đề được giao.
student_id	UUID	FK, NN	Sinh viên; xoá tài khoản cascade lượt.
question_set	JSONB	NN	Snapshot câu hỏi an toàn của lượt thi.
answers	JSONB	NN	Câu trả lời theo ID; mặc định object rỗng.
marked_questions	JSONB	NN	Danh sách câu được đánh dấu xem lại.
current_question	INTEGER	NN	Vị trí hiện tại để tiếp tục bài.
started_at	TIMESTAMPTZ	NN	Thời điểm bắt đầu.
deadline_at	TIMESTAMPTZ	NN	Hạn hoàn thành bài.
submitted_at	TIMESTAMPTZ	NULL	Thời điểm nộp bài.
submission_id	UUID	FK, UQ, NULL	Bài nộp đã tạo cho lượt thi.
updated_at	TIMESTAMPTZ	NN	Thời điểm cập nhật gần nhất.

3.8.8 Bảng exam_attempt_events – sự kiện lượt thi

Bảng sự kiện hỗ trợ giám sát một số dấu hiệu thao tác trình duyệt. Schema hiện tại định nghĩa loại sự kiện `tab_hidden`; đây không phải bằng chứng đầy đủ về hành vi của sinh viên, cũng không thay thế cơ chế giám sát trực tiếp hoặc quy chế thi.

Bảng 3.12. Từ điển dữ liệu bảng exam_attempt_events.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh sự kiện.
attempt_id	UUID	FK, NN	Lượt thi; xoá lượt cascade sự kiện.
event_type	VARCHAR(32)	NN, CK	Loại sự kiện cho phép, hiện gồm tab_hidden.
occurred_at	TIMESTAMPTZ	NN	Thời điểm ghi nhận sự kiện.

3.8.9 Bảng submissions – bài nộp

Submission là bản ghi nộp bài có cấu trúc thống nhất cho lịch sử và chấm điểm. Câu trả lời được giữ trong JSONB; trạng thái phân biệt bài đã nộp, đang chấm hoặc đã chấm. Sinh viên có thể bị đặt NULL khi tài khoản bị xoá theo chính sách schema.

Bảng 3.13. Từ điển dữ liệu bảng submissions.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh bài nộp.
exam_id	UUID	FK, NN	Đề được nộp.
student_id	UUID	FK, NULL	Sinh viên nộp.
answers	JSONB	NN	Nội dung trả lời đã nộp.
status	VARCHAR(16)	NN, CK	submitted, grading hoặc graded.
submitted_at	TIMESTAMPTZ	NN	Thời điểm nộp.

3.8.10 Bảng results – kết quả chấm

Mỗi bài nộp có tối đa một bản ghi kết quả. Kết quả giữ điểm, phản hồi và thời điểm chấm. Mức điểm tối đa được xác định từ metadata của đề và được kiểm tra trong luồng chấm.

Bảng 3.14. Từ điển dữ liệu bảng results.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh kết quả.
submission_id	UUID	FK, NN, UQ	Bài nộp; một kết quả cho mỗi bài.
score	NUMERIC(5,2)	NN	Điểm số, mặc định 0.
feedback	TEXT	NULL	Nhận xét của người chấm.
graded_at	TIMESTAMPTZ	NN	Thời điểm ghi/cập nhật điểm.

3.8.11 Bảng rubrics – tiêu chí chấm

Bảng `rubrics` gắn bộ tiêu chí với đề thi. Tiêu chí có dạng JSONB để mô tả cấu trúc rubric mà không tạo bảng con cho từng cấp điểm. Việc lưu rubric không đồng nghĩa đã có công cụ AI tự động áp dụng rubric.

Bảng 3.15. Từ điển dữ liệu bảng rubrics.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh bộ rubric.
exam_id	UUID	FK, NN	Đề áp dụng; xoá đề cascade rubric.
title	VARCHAR(255)	NN	Tên tiêu chí chấm.
criteria	JSONB	NN	Danh sách tiêu chí/mức điểm; mặc định mảng rỗng.

3.8.12 Bảng classroom_rooms – lớp học trực tuyến

Lớp học lưu thông tin buổi học, chế độ kết nối và quyền nhận thành viên. Mã tham gia là duy nhất; tài khoản tạo lớp là bắt buộc. Google Meet URL được kiểm tra ở API khi chế độ kết nối là Meet.

Bảng 3.16. Từ điển dữ liệu bảng classroom_rooms.

Trường	Kiểu	Ràng buộc	Ý nghĩa
id	UUID	PK	Định danh lớp.
name	VARCHAR(160)	NN	Tên lớp học.
description	TEXT	NN	Mô tả, mặc định chuỗi rỗng.
mode	VARCHAR(20)	NN, CK	google_meet hoặc direct.
meet_url	TEXT	NULL	Liên kết Google Meet nếu chế độ tương ứng.
access_code	VARCHAR(8)	NN, UQ	Mã tham gia lớp duy nhất.
status	VARCHAR(16)	NN, CK	scheduled, live hoặc ended.
admissions_open	BOOLEAN	NN	Có nhận sinh viên mới hay không.
started_at	TIMESTAMPTZ	NULL	Thời điểm bắt đầu buổi học.
ended_at	TIMESTAMPTZ	NULL	Thời điểm kết thúc buổi học.
created_by	UUID	FK, NN	Giảng viên/quản trị viên tạo lớp.
created_at	TIMESTAMPTZ	NN	Thời điểm tạo.

3.8.13 Bảng classroom_members – thành viên lớp

Bảng nối thành viên có khoá ghép để cùng một sinh viên không gia nhập trùng một lớp. Lịch sử gia nhập được lưu theo thời gian.

Bảng 3.17. Từ điển dữ liệu bảng classroom_members.

Trường	Kiểu	Ràng buộc	Ý nghĩa
classroom_id	UUID	PK, FK	Lớp học; xoá lớp cascade thành viên.
student_id	UUID	PK, FK	Tài khoản sinh viên; xoá tài khoản cascade quan hệ.
joined_at	TIMESTAMPTZ	NN	Thời điểm tham gia.

3.9 Danh mục API theo nhóm nghiệp vụ

Các endpoint dưới đây được tổng hợp từ route và tài liệu API trong dự án. API có tiền tố `/api`; các endpoint đánh dấu "cần xác thực" yêu cầu JWT, còn thao tác quản trị/giảng viên có kiểm tra vai trò. Hợp đồng HTTP có thể được mô tả bằng OpenAPI để tạo tài liệu và kiểm thử nhất quán [10].

3.9.1 Xác thực, tài khoản và ngân hàng câu hỏi

Bảng 3.18. Endpoint xác thực, người dùng và câu hỏi.

Method	Endpoint	Mục đích / quyền
POST	/auth/register	Đăng ký tài khoản sinh viên.
POST	/auth/login	Đăng nhập bằng email/mật khẩu.
GET	/auth/me	Lấy hồ sơ/token mới; cần xác thực.
PUT	/auth/profile	Cập nhật hồ sơ tài khoản; cần xác thực.
PUT	/auth/password	Đổi mật khẩu; cần xác thực.
GET	/auth/google	Khởi tạo đăng nhập Google nếu được cấu hình.
GET	/auth/google/callback	Nhận kết quả chuyển hướng OAuth.
GET	/users	Danh sách tài khoản/phân trang; admin.
POST	/users	Tạo tài khoản quản trị; admin.
GET	/questions	Tra cứu, lọc và phân trang câu hỏi.
POST/PUT/DELETE	/questions	Thêm/sửa/xoá câu hỏi; teacher/admin, xoá/sửa còn kiểm tra quyền sở hữu.

3.9.2 Tài liệu và quản lý đề thi

Bảng 3.19. Endpoint xử lý tài liệu và vòng đời đề thi.

Method	Endpoint	Mục đích / quyền
POST	/documents/upload	Tải và trích xuất tài liệu; teacher/admin.
POST	/documents/analyze	Phân tích nội dung và trả ứng viên câu hỏi; teacher/admin.
POST	/exams/generate	Tạo đề từ câu hỏi đã chọn; teacher/admin.
GET	/exams	Danh sách đề theo phạm vi người dùng.
GET	/exams/:id	Chi tiết đề; câu trả về cho sinh viên ẩn đáp án nội bộ.
PUT	/exams/:id	Cập nhật metadata/trạng thái đề.
DELETE	/exams/:id	Xoá đề theo quyền.
POST	/exams/:id/variants	Tạo mã đề và ánh xạ lại đáp án.
POST	/exams/:id/validate	Kiểm tra cấu trúc/ma trận đề.

3.9.3 Phòng thi, bài nộp và lớp học

Bảng 3.20. Endpoint tổ chức thi, chấm bài và quản lý lớp học.

Method	Endpoint	Mục đích / quyền
POST	/exam-rooms	Teacher/admin tạo phòng và mã tham gia.
GET	/exam-rooms	Teacher/admin liệt kê phòng được quản lý.
PATCH	/exam-rooms/:id/status	Mở/đóng phòng; kiểm tra chủ sở hữu.
GET	/exam-rooms/:id/attempts	Giám sát tiến độ và sự kiện; teacher/admin.
GET	/exam-rooms/:id/grading	Lấy các lượt thi đã nộp để chấm.
POST	/exam-rooms/:code/start	Sinh viên bắt đầu/tiếp tục bài bằng mã phòng.
PUT	/exam-rooms/attempts/:id/answers	Lưu câu trả lời và vị trí; chủ lượt thi.
POST	/exam-rooms/attempts/:id/submit	Nộp bài, tự chấm trắc nghiệm.
POST	/exam-rooms/attempts/:id/grade	Teacher/admin lưu điểm/phản hồi tự luận.
GET	/exam-rooms/my-attempts	Sinh viên xem lịch sử dự thi.
POST	/classrooms	Teacher/admin tạo lớp học.
POST	/classrooms/join	Sinh viên gia nhập bằng mã lớp.
GET/PATCH	/classrooms/:id	Xem lớp hoặc đổi trạng thái/nhận thành viên theo quyền.

3.10 Danh mục migration dữ liệu

Các migration trong dự án ghi nhận việc mở rộng schema theo chức năng. Bảng 3.21 tóm tắt từ tên file; trước khi chạy trên dữ liệu thật cần đọc nội dung migration, xác minh thứ tự, sao lưu và có phương án rollback.

Bảng 3.21. Danh mục migration hiện diện trong dự án.

Migration	Phạm vi thể hiện ở tên/tệp	Nhóm dữ liệu
001	initial_schema	Schema ban đầu: người dùng, câu hỏi, đề và dữ liệu liên quan.
002	add_metadata_to_exams	Metadata cấu hình cho đề thi.
003	exam_generation_validation	Trường phục vụ kiểm tra sinh đề.
004	add_answer_text_to_questions	Đáp án dạng văn bản cho câu hỏi.
005	add_question_source_evidence	Bằng chứng trích từ tài liệu nguồn.
006	add_essay_cognitive_rubric	Mức độ nhận thức và rubric câu tự luận.
007	online_exam_rooms	Phòng thi, lượt thi trực tuyến và sự kiện.
008	user_profiles	Trường hồ sơ tài khoản.
009	university_user_profiles	Mã và thuộc tính hồ sơ đại học.
010	exam_tab_switch_events	Lịch sử sự kiện chuyển tab.
011	classrooms	Lớp học, thành viên và mã tham gia.
012	classroom_lifecycle	Trạng thái vòng đời và nhận thành viên lớp.

3.11 Ma trận kiểm thử bổ sung

Bên cạnh 35 ca tự động đã chạy, Bảng 3.22 là bộ kịch bản kiểm thử thủ công đề xuất cho buổi trình diễn. Đây không phải kết quả đã chạy; cột trạng thái được để trống để người nộp cập nhật sau khi thực hiện.

Bảng 3.22. Kịch bản kiểm thử thủ công trước khi bảo vệ.

Mã	Bước kiểm thử	Kết quả mong đợi	Trạng thái ghi
M01	Đăng nhập lần lượt bằng vai trò sinh viên và giảng viên.	Menu và endpoint phù hợp vai trò; quyền không hợp lệ bị từ chối.	Chưa nhận
M02	Tải PDF có văn bản thật và kiểm tra phần text trích xuất.	Nội dung có thể xem trước; câu hỏi liên kết được với đoạn nguồn.	Chưa nhận
M03	Tải PDF quét chất lượng thấp hoặc file rỗng.	Có cảnh báo trích xuất; không sinh đề thành công giả.	Chưa nhận
M04	Tạo đề ma trận vượt số câu hiện có.	Hiện rõ số câu thiếu; không phát hành đề không hợp lệ.	Chưa nhận
M05	Gửi duyệt và chuyển trạng thái đề.	Trạng thái lưu sau khi tải lại trang.	Chưa nhận
M06	Vào phòng thi, trả lời một phần, tải lại rồi tiếp tục.	Đáp án/vị trí được khôi phục từ đúng lượt.	Chưa nhận
M07	Nộp bài có trắc nghiệm và tự luận.	Trắc nghiệm được chấm; tự luận chờ giảng viên chấm.	Chưa nhận
M08	Sinh viên thử xem API chi tiết đề/đáp án nội bộ.	API không trả đáp án đúng của bài thi.	Chưa nhận
M09	Gia nhập lớp đã kết thúc hoặc đóng nhận thành viên.	Từ chối người mới; thành viên đã có vẫn theo chính sách.	Chưa nhận
M10	Mở giao diện trên thiết bị di động, bàn phím và phóng đại.	Bố cục và thao tác chính còn sử dụng được.	Chưa nhận

3.12 Rủi ro, giới hạn và biện pháp giảm thiểu

Bảng 3.23. Sổ đăng ký rủi ro của nguyên mẫu AI ExamHub DAU.

Rủi ro	Tác động	Biện pháp đề xuất
Đáp án AI sai	Sinh viên nhận nội dung sai; đề thi mất độ tin cậy.	Bắt buộc giảng viên xác minh câu, đáp án và nguồn trước phát hành.
Prompt injection từ tài liệu	Nội dung tải lên có thể chứa chỉ dẫn làm lệch yêu cầu cho mô hình.	Cô lập tài liệu khỏi instruction, loại hành động công cụ nhạy cảm, kiểm tra đầu ra và quyền duyệt của người.
Lộ dữ liệu qua API LLM	Học liệu/tài liệu có dữ liệu cá nhân đi ra nhà cung cấp.	Cảnh báo trước khi gửi, lọc dữ liệu nhạy cảm, cấu hình chính sách lưu giữ và nhà cung cấp được phép.
Lộ đáp án thi	Lộ đáp án làm mất công bằng kiểm tra.	Backend chỉ trả question view đã loại answer fields; test truy cập trực tiếp bằng vai trò sinh viên.
Chiếm đoạt tài khoản	Truy cập hồ sơ, đề, kết quả và dữ liệu lớp.	Secret mạnh, HTTPS, xác thực phù hợp, giới hạn quyền, nhật ký và quy trình đổi/thu hồi token.
Tệp độc hại/quá lớn	Cạn bộ nhớ, lỗi parser hoặc ảnh hưởng dịch vụ.	Giới hạn kích thước/định dạng, kiểm tra nội dung, cách ly xử lý và giới hạn tài nguyên.
OCR tiếng Việt kém	Nội dung nguồn bị nhận sai rồi lan truyền sang câu hỏi.	Định cấu hình ngôn ngữ phù hợp, hiển thị nguồn, đo lỗi OCR và cho phép người dùng hiệu chỉnh.
Database development	Mất dữ liệu hoặc lộ mật khẩu do cấu hình mặc định.	Thay mật khẩu, tách network, sao lưu/khôi phục kiểm chứng và dùng secret manager.

Sổ đăng ký là phân tích rủi ro thiết kế, không phải báo cáo kiểm thử xâm nhập. Các rủi ro prompt injection và phân quyền dịch vụ API cần được đối chiếu với khuyến nghị OWASP về ứng dụng web, API và LLM [14, 15, 16]; khung NIST giúp tổ chức việc nhận diện, đo và xử lý rủi ro AI theo vòng đời [8].

3.13 Mục tự kiểm trước khi nộp

- [ ] Đã thay tên, MSSV, lớp, giảng viên hướng dẫn và năm học trên bìa.
- [ ] Đã rà soát mọi chương, mục lục, danh mục hình/bảng và số tham chiếu.
- [ ] Đã kiểm tra lại nội dung PDF/DOCX/OCR, các hạn chế kỹ thuật và phiên bản code được dùng để báo cáo.
- [ ] Đã chạy lại API tests, frontend build và ghi đúng phiên bản kết quả.
- [ ] Đã kiểm tra các đề xuất AI với tài liệu gốc; không coi confidence là xác suất đúng.
- [ ] Đã bổ sung ảnh chụp giao diện thật nếu giảng viên yêu cầu; các hình wireframe trong báo cáo được nhận diện là hình minh hoạ.
- [ ] Đã kiểm tra và ghi nguồn cho tối thiểu 15 tài liệu tham khảo có liên kết; mọi tài liệu trích dẫn đều có trong danh mục.
- [ ] Đã thay toàn bộ cấu hình development trước khi triển khai có dữ liệu thật.
- [ ] Đã xác nhận số trang sau khi biên dịch XeLaTeX; không coi số trang ước lượng trong source là số trang PDF.
- [ ] Đã xoá nội dung hướng dẫn và placeholder không thuộc bản nộp cuối.

3.14 Nhận xét của giảng viên hướng dẫn

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Đà Nẵng, ngày ... tháng ... năm ...

Giảng viên hướng dẫn

Điểm số

Bằng số: . . . . . .

Bằng chữ: . . . . . .

Giảng viên hướng dẫn